

Bilgisayar Bilimlerine Giriş

(temel bilgiler)

Bit: Binary Digit (0 veya 1)

Bit örüntüleri ile sayısal değer, haff, dijital fotoğraf, ses vb. bilgiyi temsil etmek için kullanılır.

Boolean İşlemleri

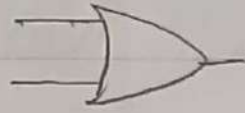
(DOĞRU/TRUE = 1, YANLIŞ/FALSE = 0)

AND (ve)

OR (veya)

XOR

NOT (Değil)



Inputs (Girizler)	Output (Çikar)
0 0	0
0 1	0
1 0	0
1 1	1

Inputs	Output
0 0	0
0 1	1
1 0	1
1 1	1

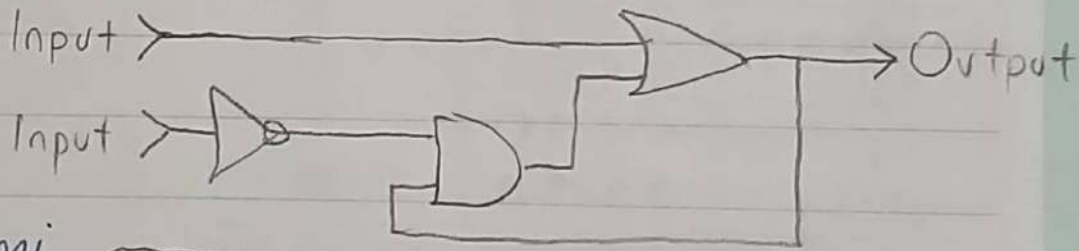
Inputs	Output
0 0	0
0 1	1
1 0	1
1 1	0

Input	Output
1	0
0	1

NOT
NAND → (AND'in tersi)
NOR → (OR'un tersi)
XNOR → (XOR'un tersi)

Yükarıdaki kapılar (gates) Boole işlemi hesaplayan donanımlardır. Bir bit depolayabilen kapılardan oluşmuş devreye **flip-flop** denir.

(flip-flop örneği) →

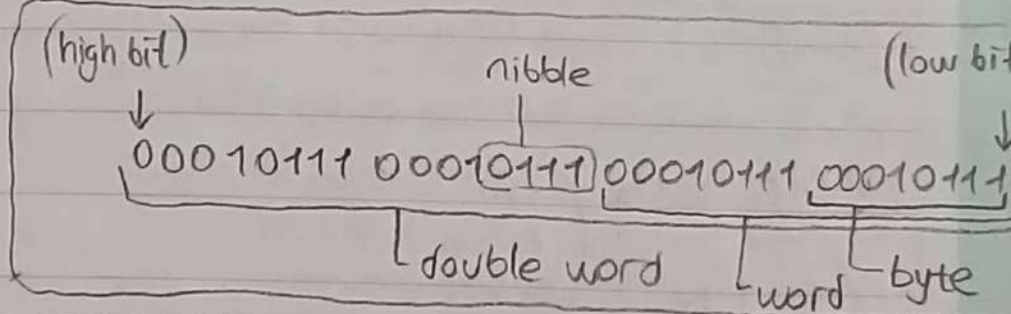


Sayıların Gösterimi

4 bits = 1 nibble

8 bits = 1 byte

16 bits = 2 byte = 1 word



1024 Byte = 1 Kb (Kilobyte)

(high → en soldaki)
(low → en sağdaki) ←

byte (b)	kilobyte (Kb)	Megabyte (MB)	Gigabyte (GB)	Terabyte (TB)	Petabyte (PB)	Exabyte (EB)
----------	---------------	---------------	---------------	---------------	---------------	--------------

(temel bilgiler)

TABANLAR ARASI DÖNÜŞÜM

$$8^3 \cdot 1 + 8^2 \cdot 5 + 8^1 \cdot 1 + 8^0 \cdot 2 = 842$$

(basamakları 0'inci kuvvetten başlayıp sayıyla çarp ve topla)

$(47)_{10} = (?)_2 \Rightarrow 47 \begin{array}{r|l} 2 & \\ \hline 46 & 23 \\ 22 & 11 \\ 11 & 5 \\ 5 & 2 \\ 2 & 1 \\ 1 & 0 \end{array}$
 $= (101111)_2 \leftarrow$ SIR

 $(1600)_{10} = (?)_{16} \Rightarrow 1600 \begin{array}{r|l} 16 & \\ \hline 1600 & 100 \\ 96 & 6 \\ 4 & \end{array}$
 $= (640)_{16}$ SIRA
SIRA

Sayı Tabanları

(temel bilgiler)

Binary: 1's Complement (1'e tümleyeni)

01011010
↓

10100101 ✓ (all bits reversed)

Binary: 2's Complement (2'ye tümleyeni)

01011010

10100101

1's comp.

↓ +1

10100110 ✓

(1 added to 1's complement)

01011010

10100110

SHORTCUT

- start from the right
- find the first 1 bit
- change every other following bits (after the first 1)

Signed

unsigned

255 → 11111111

2's complement

00000001

signed -1

most significant bit represents the sign of the number.

If it's 1, 2's complement shows the absolute value of the number.

-1 = 11111111

negative sign

(255 = 01111111)
signed

1111 1111 unsigned → 255

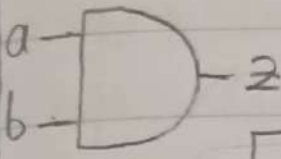
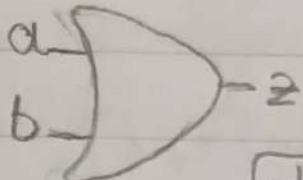
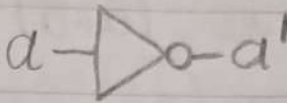
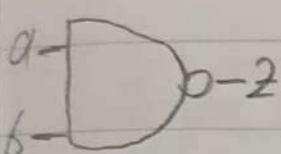
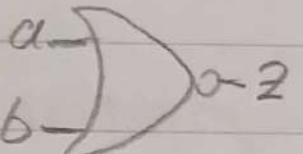
1111 1111 signed 2's comp → 0000 0001 = 1

1111 1111 signed = -1

0111 1111 signed = 127

0111 1111 signed = 255

Lojik Devreler

Ve (AND)	Veya (OR)	Değil (NOT)	Ya Da (XOR)
 	 	 	 
NAND	NOR		XNOR
			

Boole Cebirinin Aksiyom ve Teoremleri:

Lojik devrelerde belirlenen kurallar kimesinin dayanağı Boole ce
 Bütün cebirsel yapılarıdaki gibi Boole cebirinde de doğruluğu kabul edilebilir
 ve ispatlanabilir önermeler olmak üzere iki kural dizisi mevcuttur.

→ (Doğru kabul edilenler = Aksiyom ispatlanabilirler = Teorem)

Aksiyomlar

- Kullanılan sayılar: 0 ve 1 Cebirsel ifadeler: + ve •
- $1+1=1$, $0+0=0$, $0+1=1$ (← veya olarak da düşünülebilir)
- $1.1=1$, $1.0=0$, $0.0=0$

Teoremler

- $a+b = b+a$, $a.b = b.a$ (Değişim)
- $a+b+c = (a+b)+c = a+(b+c)$ (Birleşim)
- $a+b.c = (a+b).(a+c)$, $a.(b+c) = (a.b) + (b.c)$
- $a+a = a$, $a.a = a$
- $a+ab = a$, $a.(a+b) = a$, $a(1+b) = a$ $a+\bar{a}b = a+b$
- [De Morgan]: $\overline{(a+b+\dots)} = \bar{a} + \bar{b} + \bar{c} + \dots$
- [Shannon Teoremi]: $f(a,b,c) = [a + f(0,b,c)] . [\bar{a} + f(1,b,c,d)]$

(parantez kaldır)

Lojik Devreler

* $f = ab + \bar{a}c + bc$ fonksiyonunu sadeleştiririz. (QVA) ✓

$$\begin{aligned} \rightarrow ab + \bar{a}c + bc &= ab + \bar{a}c + bc(a + \bar{a}) \\ &= ab + \bar{a}c + abc + \bar{a}bc = ab \underbrace{[1+c]}_1 + \bar{a}c \underbrace{[1+b]}_1 \\ &= \boxed{ab + \bar{a}c} \end{aligned}$$

Boole Cebir Fonksiyonları

a, b, c, d (veya $x_1, x_2, x_3, \dots, x_n$) değişkenleri ve sabitleri, Boole cebirinin ve/veya tümeleme işlemleri kullanılarak oluşturulan n değişkenli f fonksiyonlarına denir.

Min Terimler & Max Terimler

X	Y	minterm	maxterm
0	0	$\bar{x}\bar{y} \quad m_0$	$x+y \quad m_0$
0	1	$\bar{x}y \quad m_1$	$x+\bar{y} \quad m_1$
1	0	$x\bar{y} \quad m_2$	$\bar{x}+y \quad m_2$
1	1	$xy \quad m_3$	$\bar{x}+\bar{y} \quad m_3$

Minterm $\rightarrow$ AND ile birleştirildiklerinde (çarpıldıklarında) 1 değeri verir.

Maxterm $\rightarrow$ OR ile birleştirildiğinde (toplandıklarında) 0 değeri verir.

$\rightarrow$ Minimum terimler kanonik biçimde gösterilir. $f(a,b,c) = \bar{a}\bar{b}c + \bar{a}b\bar{c}$
(Çarpımlar toplamı (SOP))

$$f(a,b) = a + b \rightsquigarrow \begin{array}{cc|c} a & b & f \\ \hline 0 & 0 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{array} \begin{array}{l} m_0 \\ m_1 \\ m_2 \\ m_3 \end{array} \quad f = \boxed{\bar{a}b + a\bar{b} + ab}$$

(optimizasyon için kullanılır.)
 $\downarrow$
 $\rightarrow a + b$

(Toplamlar Çarpımı (POS))

$$f(a,b) = a \cdot b \rightsquigarrow \begin{array}{cc|c} a & b & f \\ \hline 0 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \end{array} \begin{array}{l} m_0 \\ m_1 \\ m_2 \\ m_3 \end{array} \quad f = \boxed{(a+b)(a+\bar{b})(\bar{a}+b)}$$

Shannon Teoremi Örneği

$$\begin{aligned} f(a,b,c) &= \bar{a}\bar{b}\bar{c} \cdot f(0,0,0) + \bar{a}\bar{b}c \cdot f(0,0,1) + \bar{a}b\bar{c} \cdot f(0,1,0) + \bar{a}bc \cdot f(0,1,1) \\ &+ a\bar{b}\bar{c} \cdot f(1,0,0) + a\bar{b}c \cdot f(1,0,1) + ab\bar{c} \cdot f(1,1,0) + abc \cdot f(1,1,1) \end{aligned}$$

(Shannon A) (minterm)

Lojik Devreler

* $f = ab + \bar{a}c + bc$ fonksiyonunu sadeleştiririz. (AVA) $\Rightarrow$

$$\begin{aligned} \rightarrow ab + \bar{a}c + bc &= ab + \bar{a}c + bc(a + \bar{a}) \\ &= ab + \bar{a}c + abc + \bar{a}bc = ab \underbrace{[1+c]}_1 + \bar{a}c \underbrace{[1+b]}_1 \\ &= \boxed{ab + \bar{a}c} \end{aligned}$$

Boole Cebir Fonksiyonları

a, b, c, d (veya $x_1, x_2, x_3, \dots, x_n$) değişkenleri ve sabitleri, Boole cebirinin ve/veya tümlenme işlemleri kullanılarak oluşturulan n değişkenli f fonksiyonlarına denir.

Min Terimler & Max Terimler

X	Y	minterm	maxterm
0	0	$\bar{x}\bar{y} \quad m_0$	$x+y \quad m_0$
0	1	$\bar{x}y \quad m_1$	$x+\bar{y} \quad m_1$
1	0	$x\bar{y} \quad m_2$	$\bar{x}+y \quad m_2$
1	1	$xy \quad m_3$	$\bar{x}+\bar{y} \quad m_3$

Minterm $\rightarrow$ AND ile birleştirildiklerinde (çarpıldıklarında) 1 değeri verir.

Maxterm $\rightarrow$ OR ile birleştirdiğinde (toplandıklarında) 0 değeri verir.

$\rightarrow$ Minimum terimler kanonik biçimde gösterilir. $f(a,b,c) = \bar{a}b\bar{c} + \bar{a}bc$
(Çarpımlar toplamı (SOP))

$$f(a,b) = a + b \sim \begin{array}{cc|c} a & b & f \\ \hline 0 & 0 & 0 \\ 0 & 1 & 1 \\ 1 & 0 & 1 \\ 1 & 1 & 1 \end{array} \begin{array}{l} m_0 \\ m_1 \\ m_2 \\ m_3 \end{array} \rightarrow \boxed{\bar{a}b + a\bar{b} + ab} \quad \begin{array}{l} \text{(optimizasyon} \\ \text{için kullanılır)} \\ \downarrow \\ a + b \end{array}$$

(Toplamlar Çarpımı (POS))

$$f(a,b) = a \cdot b \sim \begin{array}{cc|c} a & b & f \\ \hline 0 & 0 & 0 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \\ 1 & 1 & 1 \end{array} \begin{array}{l} m_0 \\ m_1 \\ m_2 \\ m_3 \end{array} \rightarrow \boxed{(a+b)(a+\bar{b})(\bar{a}+b)}$$

Shannon Teoremi Örnek

$$\begin{aligned} f(a,b,c) &= \bar{a}\bar{b}\bar{c} \cdot f(0,0,0) + \bar{a}\bar{b}c \cdot f(0,0,1) + \bar{a}b\bar{c} \cdot f(0,1,0) + \bar{a}bc \cdot f(0,1,1) \\ &+ a\bar{b}\bar{c} \cdot f(1,0,0) + a\bar{b}c \cdot f(1,0,1) + ab\bar{c} \cdot f(1,1,0) + abc \cdot f(1,1,1) \end{aligned} \quad \begin{array}{l} \text{(Shannon Al} \\ \text{goritması)} \end{array}$$

Lojik Devreler

(Shannon B) $\rightarrow f(x_1, x_2, \dots, x_n) = [x_1 + f(0, x_2, \dots, x_n)] [\bar{x}_1 + f(1, x_2, \dots, x_n)] \dots$
 $f(a, b, c) = [a + b + c + f(0, 0, 0)] [a + b + \bar{c} + f(0, 0, 1)] [a + \bar{b} + c + f(0, 1, 0)] [a + \bar{b} + \bar{c} + f(0, 1, 1)]$
 $[\bar{a} + b + c + f(1, 0, 0)] [\bar{a} + b + \bar{c} + f(1, 0, 1)] [\bar{a} + \bar{b} + c + f(1, 1, 0)] [\bar{a} + \bar{b} + \bar{c} + f(1, 1, 1)]$

$f(a, b, c) = \bar{a}b + ac$ minterm bulalım:

a	b	c	f	minterm
0	0	0	0	
0	0	1	1	m_1
0	1	0	0	
0	1	1	1	m_3
1	0	0	0	
1	0	1	0	
1	1	0	1	m_6
1	1	1	1	m_7

$$\bar{a}bc + \bar{a}b\bar{c} + ab\bar{c} + abc$$

minterm yazılımı (uzun)

$$\bar{a}b + ac \text{ (kisa)}$$

maxterm yazılımı

$$(a+b+c)(a+\bar{b}+c)(\bar{a}+b+c)(\bar{a}+b+\bar{c})$$

maxterm yazılımı (uzun)

Kanonical Biçimler Arası Dönüşüm

$$f(a, b, c) = \sum (1, 3, 6, 7) = m_1 + m_3 + m_6 + m_7$$

$$\overline{f(a, b, c)} = \sum (0, 2, 4, 5) = m_0 + m_2 + m_4 + m_5$$

$$[f(a, b, c)] = \sum (1, 3, 6, 7) = (m_0 + m_2 + m_4 + m_5)$$

$$\bar{m}_i = M_i$$

$$\bar{m}_0 + \bar{m}_2 + \bar{m}_4 + \bar{m}_5 = m_1 \cdot m_3 \cdot m_6 \cdot m_7 = \prod (1, 3, 6, 7)$$

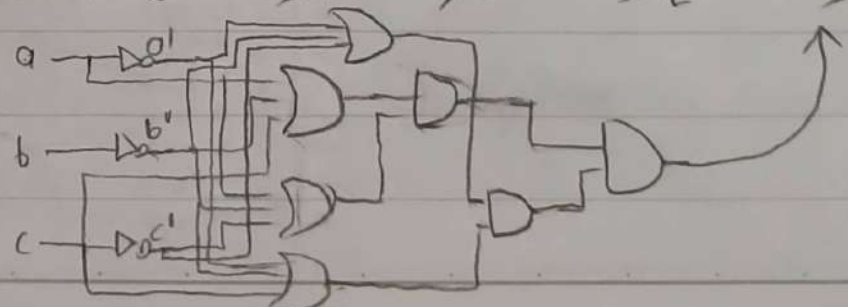
örnek

a	b	c	f	
0	0	0	1	m_0
0	0	1	1	
0	1	0	0	M_2
0	1	1	0	
1	0	0	1	
1	0	1	1	m_3
1	1	0	0	
1	1	1	0	M_7

$$f(a, b, c) = ?$$

$$f(a, b, c) = \bar{a}\bar{b}\bar{c} + \bar{a}\bar{b}c + a\bar{b}\bar{c} + a\bar{b}c$$

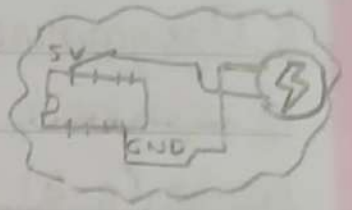
$$f(a, b, c) = (a + \bar{b} + c)(a + \bar{b} + \bar{c})(\bar{a} + \bar{b} + c)(\bar{a} + \bar{b} + \bar{c})$$



Lojik Devreler

Lojik Fonksiyon İndirgenmesi

Fazla çip kullanımı maliyeti artırır.



Yöntemleri:

- 1- Gözle Dayalı
- 2- Karnaugh Diyagramları
- 3- Φ

1. Yöntem Örneler HATIRLATILIM

$$f(a,b,c,d) = \bar{a}b\bar{c} + ad + b\bar{c}\bar{d} = b\bar{c}(\bar{a} + \bar{d}) + ad = b\bar{c}(\overline{ad}) + ad = \boxed{b\bar{c} + ad}$$

$$(2. yol \rightarrow) b\bar{c}(\overline{ad}) + ad = (ad + b\bar{c})(ad + \bar{a}\bar{d}) = \boxed{ad + b\bar{c}} \quad (a + (bc) = (a+b)(a+c))$$

$$f(x,y,z) = \bar{x}\bar{y}\bar{z} + x\bar{y}\bar{z} = \bar{y}\bar{z}(\bar{x} + x) = \boxed{\bar{y}\bar{z}}$$

Karnaugh Diyagramı

x	y	f
0	0	m ₀
0	1	m ₁
1	0	m ₂
1	1	m ₃

x \ y	0	1
0	m ₀	m ₁
1	m ₂	m ₃

x	y	z	f
0	0	0	m ₀
0	0	1	m ₁
0	1	0	m ₂
0	1	1	m ₃
1	0	0	m ₄
1	0	1	m ₅
1	1	0	m ₆
1	1	1	m ₇

xy \ z	0	1
00	m ₀	m ₁
01	m ₂	m ₃
11	m ₆	m ₇
10	m ₄	m ₅

3 değişken

2 değişken

(Gray-code format: always 1-bit change per step.)

xy \ zt	00	01	11	10
00	m ₀	m ₁	m ₃	m ₂
01	m ₄	m ₅	m ₇	m ₆
11	m ₁₂	m ₁₃	m ₁₅	m ₁₄
10	m ₈	m ₉	m ₁₁	m ₁₀

xyzt	f
0000	m ₀
0001	m ₁
0010	m ₂
0011	m ₃
0100	m ₄
0101	m ₅
0110	m ₆
0111	m ₇
1000	m ₈
1001	m ₉
1010	m ₁₀
1011	m ₁₁
1100	m ₁₂
1101	m ₁₃
1110	m ₁₄
1111	m ₁₅

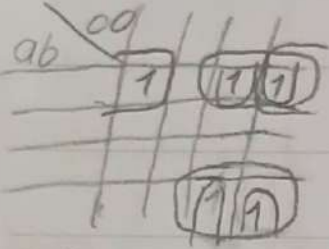
x \ yz	00	01	11	10
0	m ₀	m ₂	m ₆	m ₄
1	m ₁	m ₃	m ₇	m ₅

4 değişken

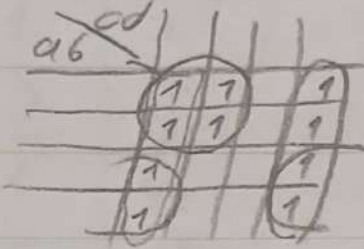
Lojik Devreler

K. Diyagramında Komşuluk Kavramı:

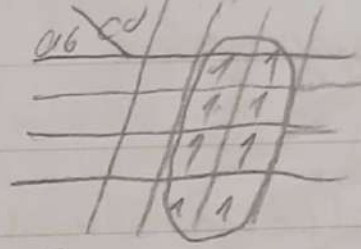
n değişkenli bir fonksiyonda 2^k tane ($k=1,2,3,\dots$) terim
kısak ($n-k$) garpanlı tek bir terime dönüşürse
bu 2^k tane terim k . dereceden komşudur.



$2^1 \Rightarrow 1$. dereceden
komşu



$2^2 \Rightarrow 2$. dereceden
komşu



$2^3 \Rightarrow 8$. dereceden
komşu

$\rightarrow$ Daha büyük gruplandırma indirgemeyi artırır.

ab \ cd	00	01	11	10
00	1	1	1	1
01	1	1	1	1
11	1	1	1	1
10	1	1	1	1

(minterm)
 $\bar{a}bcd$
X

ab \ cd	00	01	11	10
00	1	1	1	1
01	1	1	1	1
11	1	1	1	1
10	1	1	1	1

$\bar{a}b$
✓

[YERLEŞTİRME]

$$f(a,b,c,d) = a'b'c' + b'cd' + a'bcd' + ab'c'$$

Mux yöntemi
versiyonu
ÖDEV

ab \ cd	00	01	11	10
00	1	1	0	1
01	0	0	0	1
11	0	0	0	0
10	1	1	0	1

örnek $\rightarrow a'b'c'$
 $\rightarrow a'b'$ yani 00'lı satır
 $\rightarrow c'$ yani 00 ve 01 sütunları
 $\rightarrow$ sağlayan kutular 1 olarak işaretlenir.

$$\bar{b}\bar{c} + \bar{a}c\bar{d} + \bar{b}c\bar{d} \rightarrow \text{En sade hali}$$

$$0\text{'lar (maxterm) ile çözüm} \rightarrow (b+c)(c'+d')(a'+b')$$

Aşağıdaki durumda

ab \ cd	00	01	11	10
00	1	1	1	1
01	1	1	1	1
11	1	1	1	1
10	1	1	1	1

Daha büyük grup seçmek mümkün
olsa bile herhangi bir avantajı yoktur.

Lojik Devreler

K. Diyagramında Komşuluk Kavramı:

n değişkenli bir fonksiyonda 2^k tane ($k=1,2,3,\dots$) terim
kısıp $(n-k)$ garpanlı tek bir terime dönüşürse
bu 2^k tane terim k . dereceden komşudur.

ab \ cd	00	01	11	10
00	1	1	1	1
01	1	1	1	1
11	1	1	1	1
10	1	1	1	1

$2^1 \Rightarrow 1$. dereceden
komşu

ab \ cd	00	01	11	10
00	1	1	1	1
01	1	1	1	1
11	1	1	1	1
10	1	1	1	1

$2^2 \Rightarrow 2$. dereceden
komşu

ab \ cd	00	01	11	10
00	1	1	1	1
01	1	1	1	1
11	1	1	1	1
10	1	1	1	1

$2^3 \Rightarrow 8$. dereceden
komşu

$\rightarrow$ Daha büyük gruplandırma indirgemeyi artırır.

ab \ cd	00	01	11	10
00	1	1	1	1
01	1	1	1	1
11	1	1	1	1
10	1	1	1	1

(minterm)
 $\bar{a}bcd$
X

ab \ cd	00	01	11	10
00	1	1	1	1
01	1	1	1	1
11	1	1	1	1
10	1	1	1	1

$\bar{a}b$
✓

[YERLEŞTİRME]

$$f(a,b,c,d) = a'b'c' + b'cd' + a'bcd' + ab'c'$$

Mux yöntemi
versiyonu
BDEV

ab \ cd	00	01	11	10
00	1	1	0	1
01	0	0	0	1
11	0	0	0	0
10	1	1	0	1

örnek $\rightarrow a'b'c'$
 $\rightarrow a'b'$ yani 00'lı satır
 $\rightarrow c'$ yani 00 ve 01 sütunları
 $\rightarrow$ sağlayan kutular 1 olarak işaretlenir.

$$\bar{b}\bar{c} + \bar{a}c\bar{d} + \bar{b}c\bar{d} \rightarrow \text{En sade hali}$$

$$0\text{'lar (maxterm) ile çözüm} \rightarrow (b+c)(c+d')(a'+b')$$

Aşağıdaki durumda

ab \ cd	00	01	11	10
00	1	1	1	1
01	1	1	1	1
11	1	1	1	1
10	1	1	1	1

Daha büyük grup seçmek mümkün
olsa bile herhangi bir avantajı yoktur.

Lojik Devreler

③ Quine - Mc Cluskey Yöntemi

$$f(a,b,c,d) = \sum_1 (3, 7, 11, 12, 13, 14, 15)$$

→ minterm	ikili taban karşılığı	Lojik "1" sayısı
3	0011	2
7	0111	3
11	1011	3
12	1100	2
13	1101	3
14	1110	3
15	1111	4

→ Lojik "1" sayısına göre grupla. → Farklı grupları karşılaştır. (Ardışık) (1 değişimli)

3	0011	3,7 → 0-11	12,7 → X	3,7,11,15 → <u>11</u> cd
12	1100	3,11 → 011	12,11 → X	3,11,7,15 → <u>11</u> aynı
7	0111	3,13 → X	12,13 → 110-	12,13,14,15 → <u>11</u> ab
11	1011	3,14 → X	12,14 → 11-0	12,14,13,15 → 11-- (aynı)
13	1101	7,15 → -111		
14	1110	11,15 → 1-11		
15	1111	13,15 → 11-1		
		14,15 → 111-		

2 değişim
1 grup kalana
kadar 1 bit değişim
grubu alınır.

$$ab + cd (=f)$$

(Karmaşık fonksiyonları çözmek için algoritma düzenleme yöntemi.)

→ ÖDEV → $f(a,b,c,d) = a'b'c' + b'cd' + a'bcd' + ab'c'$

Lojik fonksiyonunu Quine - McCluskey Tablo yöntemi ile indirgeyip, pdf formatında sisteme yükleyiniz.

→ Doğruluk tablosu çizebilirsiniz (Fıkıtsız)

→ Terimleri kullanarak minterimleri elde ederiz

→ Gerisi yöntemle hallolur

Gözüm için
Sayfayı çektim →

Lojik Devreler

a	b	c	d	minterim
0	0	0	0	m_0
0	0	0	1	m_1
0	0	1	0	m_2
0	0	1	1	m_3
0	1	0	0	m_4
0	1	0	1	m_5
0	1	1	0	m_6
0	1	1	1	m_7
1	0	0	0	m_8
1	0	0	1	m_9
1	0	1	0	m_{10}
1	0	1	1	m_{11}
1	1	0	0	m_{12}
1	1	0	1	m_{13}
1	1	1	0	m_{14}
1	1	1	1	m_{15}

Terimler:

$$a'b'c'd' \rightarrow 0000 \rightarrow m_0 \text{ ve } m_1$$

$$b'cd' \rightarrow 0010 \rightarrow m_2 \text{ ve } m_{10}$$

$$a'bcd' \rightarrow 0110 \rightarrow m_6$$

$$ab'c' \rightarrow 1000 \rightarrow m_8 \text{ ve } m_9$$

$$m_0, m_1, m_2, m_6, m_8, m_9, m_{10}$$

$$f(a,b,c,d) = \sum_1(0,1,2,6,8,9,10)$$

Lojik Bit Sıralaması: $m_0=0 \rightarrow m_1, m_2, m_8=1 \rightarrow m_6, m_9, m_{10}=2$

Grup	Terim	ABCD	Yeni Gruplar >>
0	0	0000	0,1 $\rightarrow$ 000-
	1	0001	0,2 $\rightarrow$ 00-0
1	2	0010	0,8 $\rightarrow$ -000
	8	1000	1,9 $\rightarrow$ -001
	6	0110	2,6 $\rightarrow$ 0-10
2	9	1001	2,10 $\rightarrow$ -010
	10	1010	8,9 $\rightarrow$ 100-
			8,10 $\rightarrow$ 10-0

$$0,1,8,9 \rightarrow -00-$$

$$0,2,8,10 \rightarrow -0-0$$

$$0,8,1,9 \rightarrow -00- \text{ (aynı)}$$

$$0,8,2,10 \rightarrow -0-0 \text{ (aynı)}$$

$$\bar{a}\bar{c}\bar{d}$$

$$-00- \text{ ve } -0-0$$

P.I.	minterim involved	0	1	2	6	8	9	10
$\bar{B}\bar{C}$	0,1,8,9	x	(x)			x	x	
$\bar{B}\bar{D}$	0,2,8,10	x		(x)		x		(x)

kullanılmamış

$$= \bar{a}\bar{c}\bar{d} + \bar{b}\bar{c} + \bar{b}\bar{d}$$

$$(2,6)$$

Logik Devreler

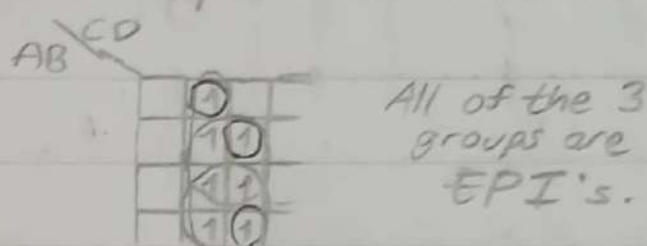
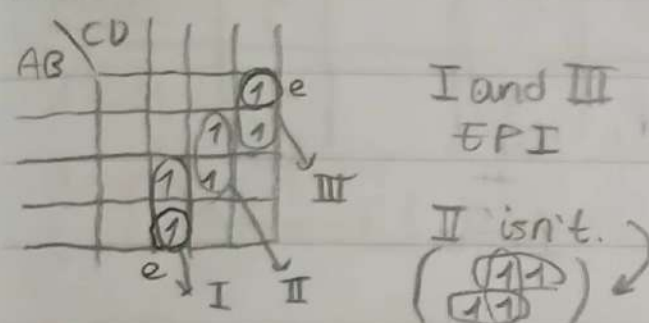
More About Karnaugh Maps (K-Maps)

→ The group of 1's is called the "implicants".

(The neighbours → 1, 2, 4, 8, ...)

→ The largest possible groups of 1 is called the "prime implicants".

→ If there is at least a single 1 which cannot be combined in any other way in a group, that group is called the "essential prime implicants". (EPI)



→ The "Don't Care" in K-Map

$$F(A,B,C) = \sum m(2,3,4,5) + \sum d(6,7) \rightarrow \text{an extra variable}$$

BC	00	01	11	10
A=0			1	1
A=1	1	1	X	X

d → 0,1 we can take it whatever we want to minimize the cost.

BC	00	01	11	10
A=0			1	1
A=1	1	1	1	1

This way, We eliminate one variable!

$$\Rightarrow \boxed{b+a} \rightarrow \bar{a}b + \bar{b}a$$

(much more simple from before.)

$$f(a,b,c,d) = \sum (3,7,11,12,15) + \sum d(0,10,13,14)$$

cd	00	01	11	10
ab=00	X		1	
ab=01			1	
ab=11	1	X	1	X
ab=10		X	1	X

$$\boxed{f = ab + cd}$$

Used X's (Don't Cares) to minimize the function.

LAB 2: K-Map

Gıkış 1 için K-Map:

a	b	c	G ₁	G ₂
0	0	0	1	1
0	0	1	0	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

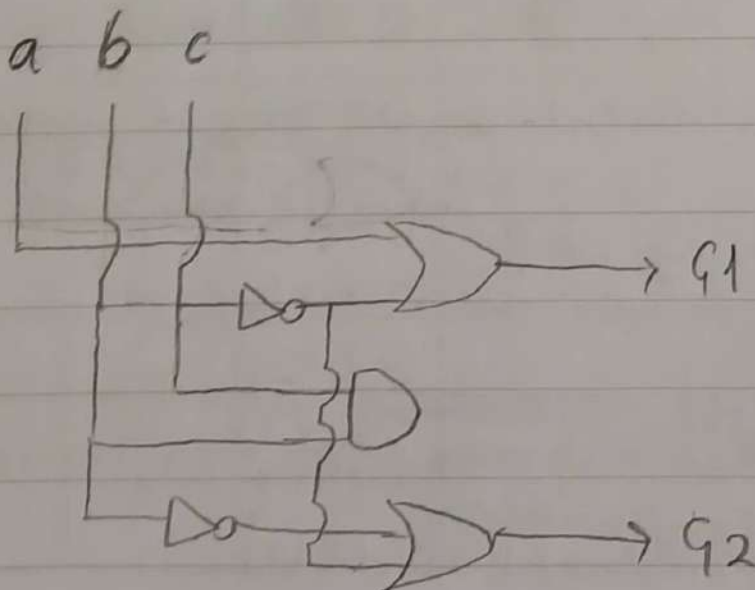
a \ bc	00	01	11	10
0	1	0	0	1
1	1	1	1	1

Gıkış 2 için K-Map:

a \ bc	00	01	11	10
0	1		1	
1	1		1	

$$G_1: f(a,b,c) = \bar{c} + a$$

$$G_2: f(a,b,c) = \bar{b}\bar{c} + bc \quad (\text{XNOR})$$

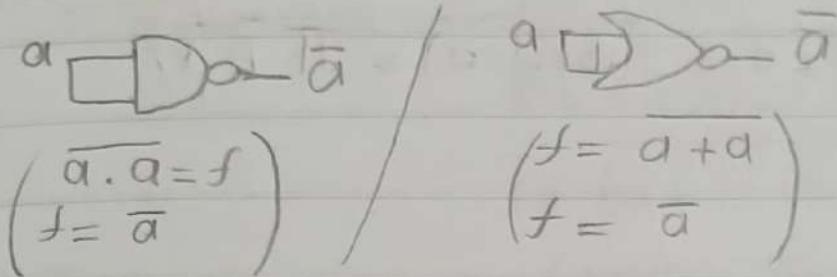


Lojik Devreler

Türetilmiş Kapılarda Temel Kapıların Yapılması

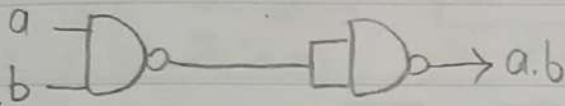
(NOR - NAND - XOR - XNOR): Türetilmiş

NOT

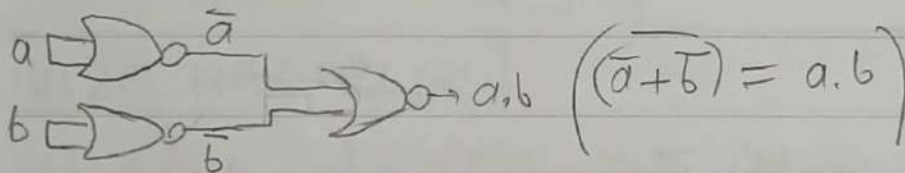


AND

$$f = a \cdot b$$

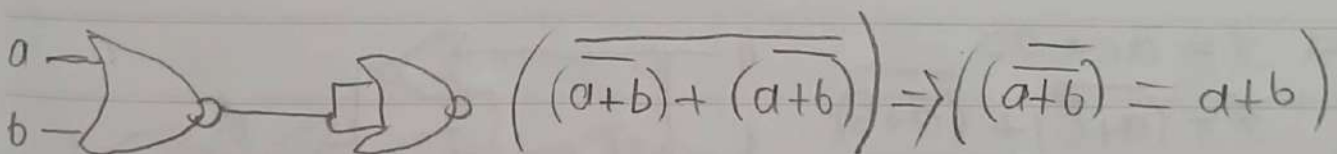
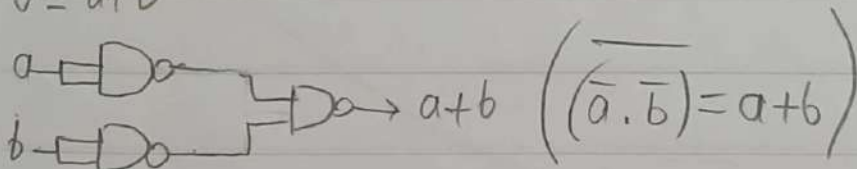


~~$$\left(\begin{array}{l} \overline{a \cdot b} \rightarrow \text{ilk kapı} \\ \overline{a \cdot b} \cdot \overline{a \cdot b} \rightarrow f \\ (\overline{a+b}) \cdot (\overline{a+b}) = f \\ (\overline{a+b}) + (\overline{a+b}) = f \end{array} \right) \rightarrow \left(\begin{array}{l} (a \cdot b) + (a \cdot b) = f \\ f = (a \cdot b) \end{array} \right) \rightarrow \left(\begin{array}{l} \text{sadeleştirme} \\ \overline{\overline{a \cdot b}} \cdot \overline{\overline{a \cdot b}} = x \\ x = (a \cdot b) \end{array} \right)$$~~



OR

$$f = a + b$$



Lojik Devreler

İndirgenmiş İfadelerin Aynı Tür Kapılarla Gerçeklenmesi

A. Çarpımlar Toplamı (NAND) Tasarımı

$$f = \sum \prod x_i \rightarrow ab + cd$$

$$\bar{f} = \overline{\sum \prod x_i} \rightarrow \overline{ab + cd} = \overline{(ab)} \cdot \overline{(cd)} = \prod \prod \bar{x}_i$$

$$\bar{f} = \prod \prod \bar{x}_i \dots$$

ÖRNEK

$f = \bar{a}\bar{b}c + \bar{a}bc + ab\bar{c} + abc$ indirgeyip NAND formunda gösterin.

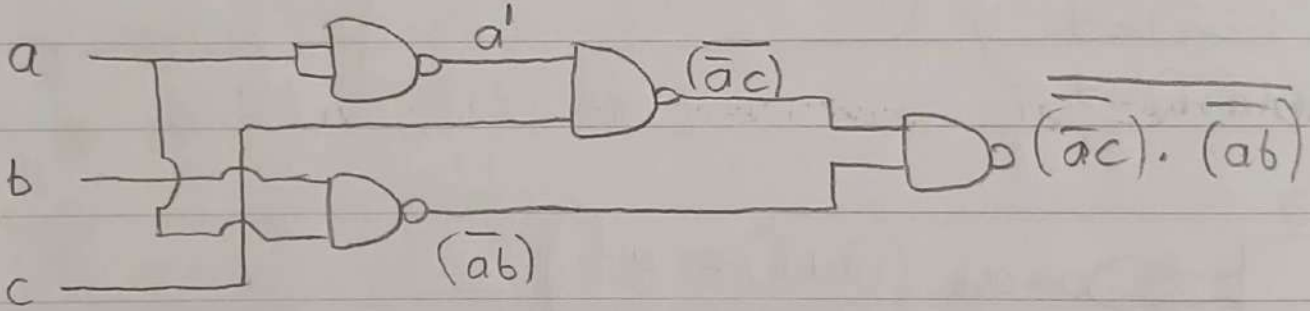
a \ bc	00	01	11	10
0		1	1	
1			1	1

$$f = \bar{a}c + ab$$

NAND

$$\bar{f} = \overline{(\bar{a}c + ab)} = \overline{(\bar{a}c)} \cdot \overline{(ab)}$$

$$f = \bar{\bar{f}} = \overline{(\overline{(\bar{a}c)} \cdot \overline{(ab)})} \rightarrow \text{NAND}$$



B. Çarpımlar Toplamı (NOR) Tasarımı

$$\prod x_i = \overline{\sum \bar{x}_i}$$

$$f = \sum \prod x_i = \sum \overline{\sum \bar{x}_i}$$

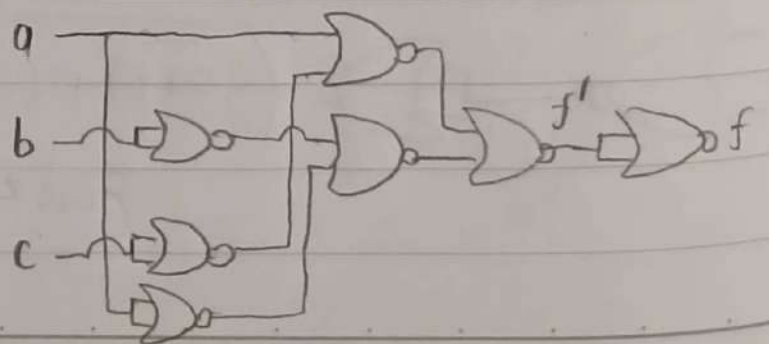
$$\bar{f} = \overline{\sum \overline{\sum \bar{x}_i}}$$

$$f = \bar{a}c + ab$$

$$f = \overline{(\overline{a+c}) + (\overline{a+b})}$$

$$\bar{f} = \overline{(\overline{a+c}) + (\overline{a+b})}$$

NOR



Lojik Devreler

C. Toplamlar Çarpımı (NAND) Tasarımı

$$f = \prod \sum x_i \quad \text{POS}_{\text{NAND}} \rightarrow (\overline{a.b})$$

$$f = \prod \prod x_i$$

$$\overline{f} = \overline{\prod \prod x_i} \rightarrow \text{NAND} \rightarrow f = (a+b).(\overline{a}+c)$$

$$f = (\overline{a.b}).(\overline{a.c})$$

$$\overline{f} = (\overline{a.b}).(\overline{a.c}) \rightarrow \text{NAND}$$

D. Toplamlar Çarpımı (NOR) Tasarımı

$$f = \prod \sum x_i \quad \text{POS}_{\text{NOR}} \rightarrow (\overline{a+b})$$

$$\overline{f} = \overline{\prod \sum x_i} = \sum \sum x_i$$

$$f = \overline{\overline{f}} = \overline{\sum \sum x_i} \rightarrow f = (a+b).(\overline{a}+c)$$

$$\overline{f} = (\overline{a+b}).(\overline{\overline{a}+c})$$

$$f = \overline{\overline{f}} = \overline{(\overline{a+b}).(\overline{\overline{a}+c})} \rightarrow \text{NOR}$$

* Kombinezional Devreler vize sonrası!

[VİZE HAZIRLIK]

$$S1) f(a,b,c,d) = \overline{a}b\overline{c} + \overline{a}b\overline{c} + \overline{a}bc + abd \quad 2 \text{ girişli TVEYA (NOR)} \\ \overline{a}(bc + \overline{b}\overline{c})$$

Lojik Devreler

S2) Aşağıda verilen K-map'te değişkenlerin bazıları boş bırakılmıştır. Gruplar toplamı şekilde indirgenmiş ifade $f = \bar{a}\bar{c}$ olacak şekilde doldur.

ab \ cd	10	?	?	?
10				
?		1	1	
?		1	1	
?				

b) oluşturduğunuz diyagrama göre $\bar{a}\bar{b}\bar{c} + \bar{a}b\bar{c} + \bar{a}bc + abd$ fonksiyonunu sadeleştirin.

cevap

ab \ cd	10	00	01	11
10				
00		1	1	
01	1	1	1	1
11			1	1

$$f = \bar{a}b + \bar{a}\bar{c} + bd$$

S3) Trafik lambasına numaralandırılmış sensörler bilgi vermektedir. Lamba kırmızı ve yeşil renk vermektedir.

S_1 $\bigcirc$ S_2 P/B Kırmızı ise K/G Yeşildir. (C'de ya da tersi)

$$I_{DB} = \bar{I}_{KG}$$

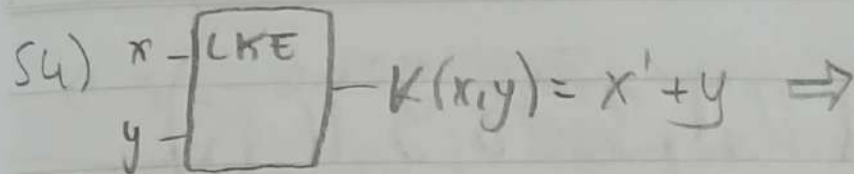
$$* S_1 \wedge S_2 = 1 \text{ ise } DB = 1$$

$$* S_1 \vee S_2 = 1 \wedge S_3 \vee S_4 = 1 \text{ ise } DB = 1$$

$$* \forall \text{Input} = 0 \text{ ise } DB = 1$$

S_1	S_2	S_3	S_4	DB	KG
0	0	0	0	1	0
0	0	0	1	0	1
0	0	1	0	0	1
0	0	1	1	0	1
0	1	0	0	1	0
0	1	0	1	1	0
0	1	1	0	0	0
0	1	1	1	0	0
1	0	0	0	1	0
1	0	0	1	1	0
1	0	1	0	1	0
1	0	1	1	1	0
1	1	0	0	1	0
1	1	0	1	1	0
1	1	1	0	1	0
1	1	1	1	1	0

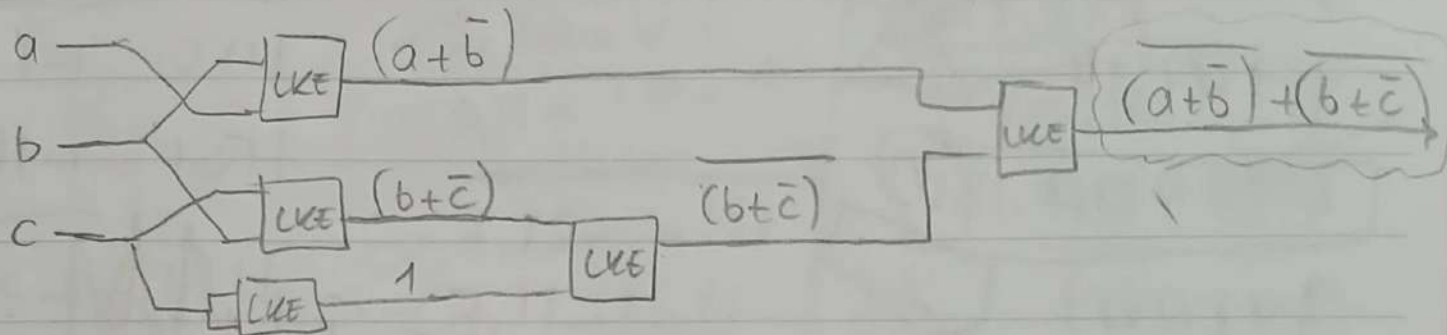
Lojik Devreler



$F(a,b,c) = \bar{a}b + \bar{b}c$
fonksiyonunu bu verilen
elementla oluşturunuz.

Cevap) $F = \bar{a}b + \bar{b}c = \overline{(a + \bar{b})} + \overline{(b + \bar{c})}$

x -  $= \underbrace{x' + x}_{11}$



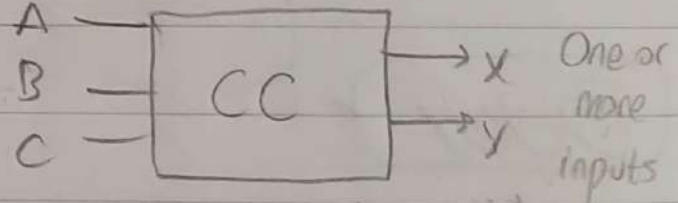
Lojik Devreler

Kombinezonasal Devre Yaklaşımı

- Decoder (Kod Gözetici)
- Encoder (Kodlayıcı)
- Selecting
- Implementing Combinational Functions Using:
 - Adder & Subtractors
 - Decoder & Encoders
 - Multiplexer & Demultiplexer
 - ROMs
 - PLAs
 - PALs
 - Lookup Tables

Kombinezonasal Devre: 0 anlık girdilere göre herhangi bir çıktı üreten devrelerdir.

(Combinational Circuit: CC)



↳ Bir hafıza verisi yok. 0 anlık +

neyse o. (Ardışıl Devrede hafıza gerekir)

Kombinezonasal Devreler NAND, NOR ve NOT ile yapılır.

(Şu ana kadarki yaptıklarımız kombinasyonel devre montajıdır.)

Combinational Logic Circuit (CLC)

Arithmetic & Logical Functions

- Adders
- Subtractors
- Comparators
- PLD's

Data Transmission

- Multiplexers
- Demultiplexers
- Encoders
- Decoders

Code Converters

- Binary
- BCD
- 7-segment

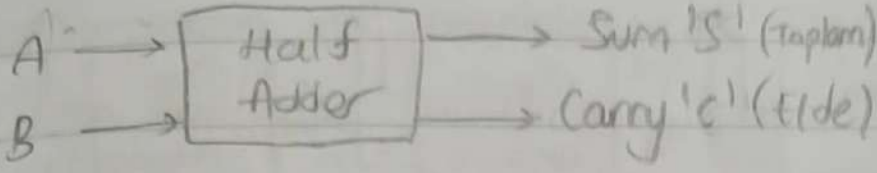
Lojik Devreler

Adder (Toplayıcı)

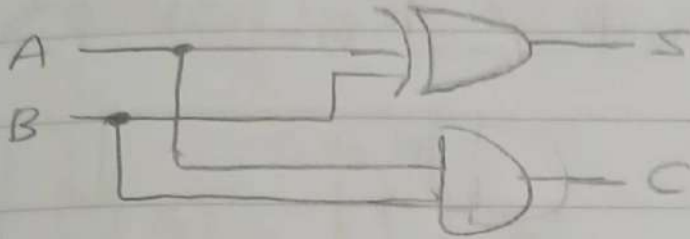
$$A \oplus B = S$$

$$A \cdot B = C$$

Half Adder: Sadece bir bitlik iki ifadeyi toplar.

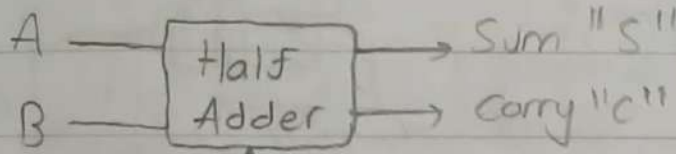


A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1



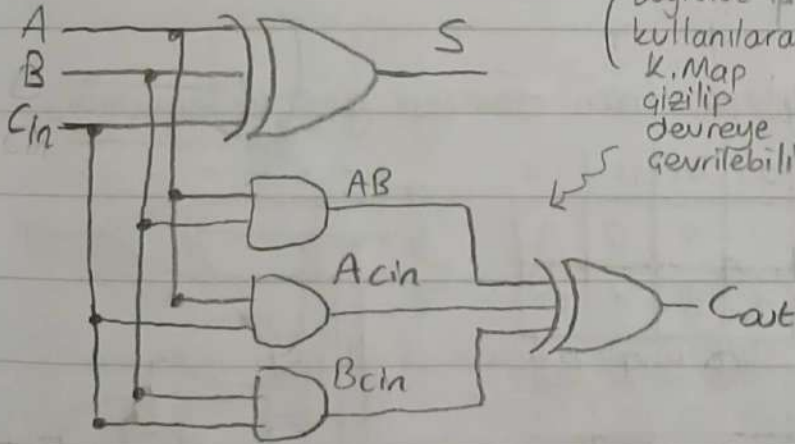
(Tam Toplayıcı)

Full Adder: İki bitlik toplama yapabilir. Bunu, Carry'yi (elde) üçüncü bir girişe ekleyerek yapar.

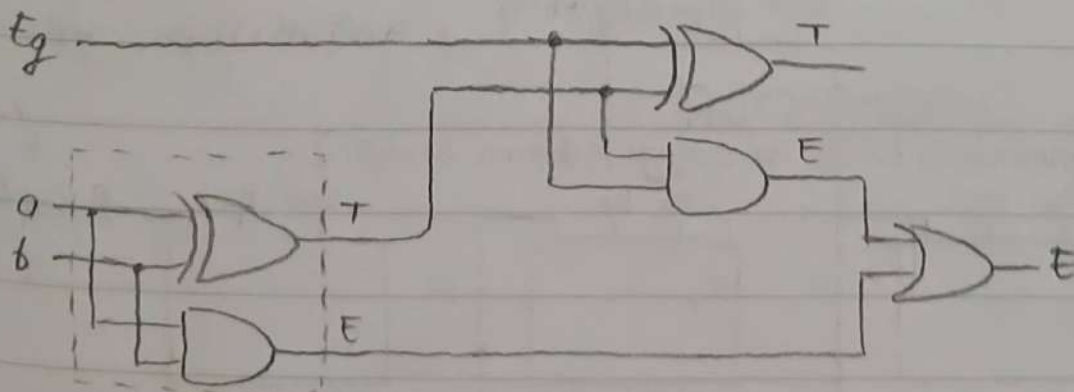


A	B	Cin	Co	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

(Doğruluk Tablosu kullanılarak K. Map çizilip devreye çevrilebilir)



Half Adder kullanarak Full Adder (Tam Toplayıcı) oluşturma:

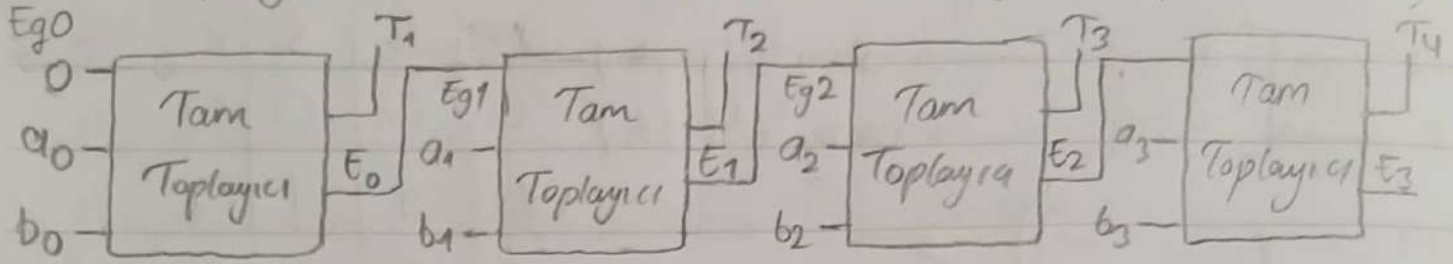


$S = T$
 (Sum / Toplam)
 $Cin = E_g$
 (Carryin = Elde girişi)
 $C = E$
 (Carry = Elde)

Lojik Devreler

PARALEL FULL ADDER

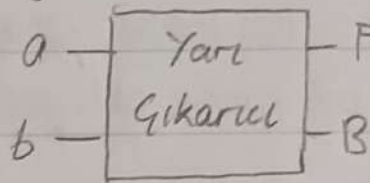
(Tam toplayıcılar kullanarak 4 bitlik iki sayının toplanması:)



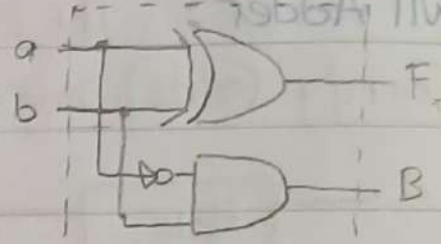
Half Subtractor (Yarı Çıkarıcı)

Çıkarma, ikinci sayının tümleyeni toplayarak gerçekleştirilebilir.

a	b	Fark (F)	Borç (B)
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0



(D=F (Difference / Fark))
(B=B (Borrow / Borç))



~Half Subtractor~

Tam Çıkarıcı (Full Subtractor)

Bu sefer girişe bir borç ekleyerek tam devreyi gerçekleyeceğiz:

X	Y	Bin	D (F)	B
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

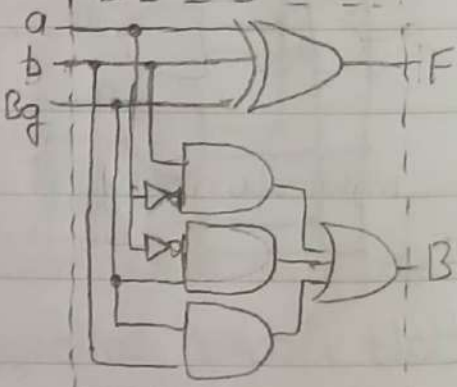
K.map

a \ bBg	00	01	11	10
0	0	1	0	1
1	1	0	1	0

a \ bBg	00	01	11	10
0	0	1	1	1
1	0	0	1	0

$$F = a \oplus b \oplus Bg$$

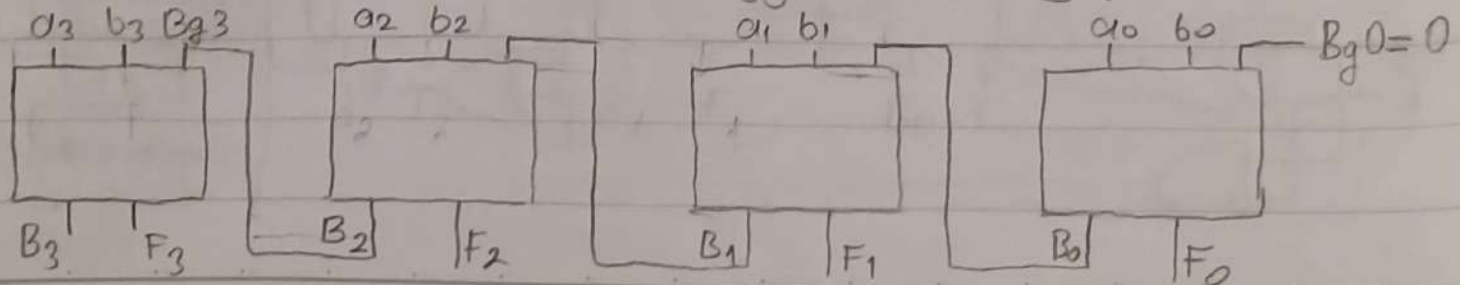
$$B = \bar{a}b + \bar{a}Bg + bBg$$



~Full Subtractor~

PARALEL FULL SUBTRACTOR

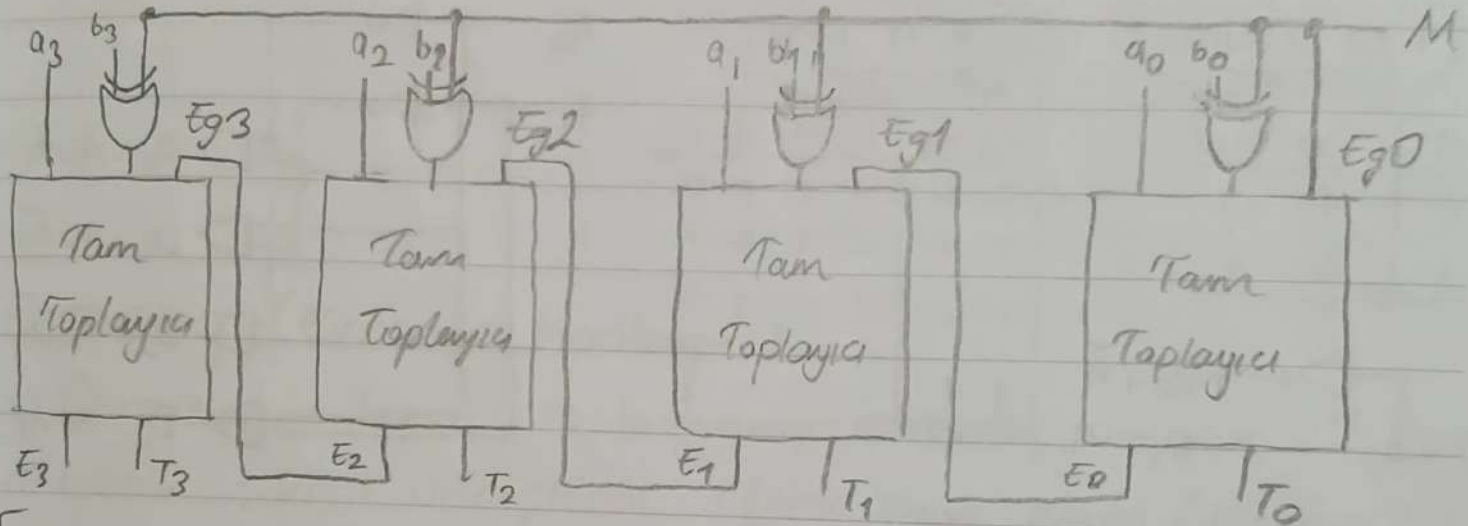
(Tam çıkarcılar kullanarak 4 bitlik iki sayıyı çıkarma örneği:)



Subtractor/Adder Combine Circuit

→ İkiye t mleyen mantığı kullanılarak toplama ve  ıkarma i lemi ni se en ve ger ekle tiren kombinezon sal devre.

→ Tam toplayıcı kullanılır,  ıkarma yapılark sa ikinci sayı ikiye t mlenir. Bunun i in bir kontrol giri i gerekir. (M)



$\{ M=0 \Rightarrow \text{Toplama} \quad M=1 \Rightarrow 1 \oplus b_0 = \overline{b_0}, E_g = 1 \xrightarrow{(2's \text{ comp.})} \text{ ıkarma} \}$

Decoder (Kod Gözetücü)

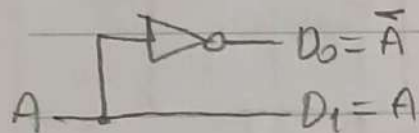
→ Belirli bitlik bir sayının binary ifadesindeki herhangi bir indisinden 1 veya 0 alan devredir.

→ Sadece tek bir bittten anlamlı çıkış almayı hedefler.

→ Giriş sayısı = n için 2^n çıkışa ihtiyaç duyar.

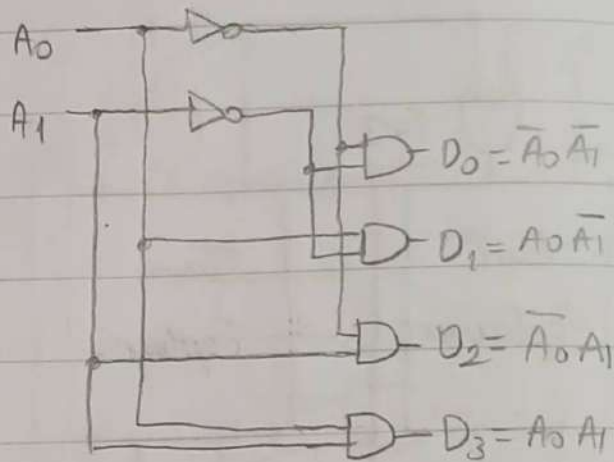
1 to 2 line Decoder

A	D_0	D_1
0	1	0
1	0	1



2 to 4 Line Decoder

A_1	A_0	D_0	D_1	D_2	D_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1



Decoder Expansion

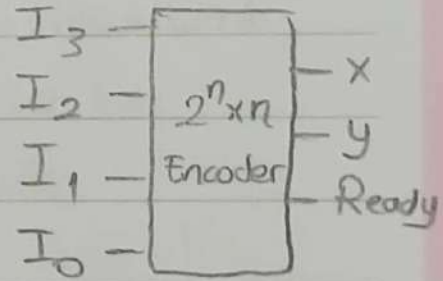
→ Daha az decoder kullanarak daha yüksek bit ifade edilebilir:

Encoding (Encoder: Kodlayıcı)

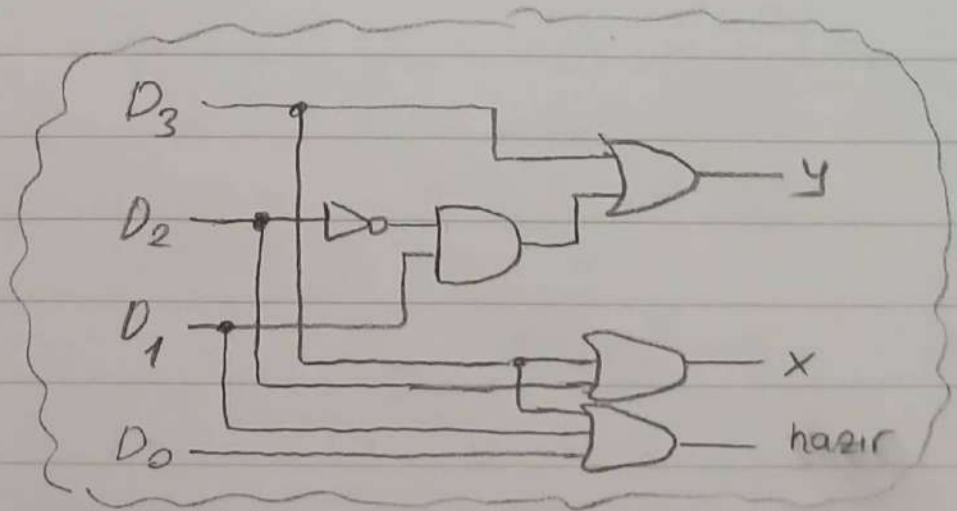
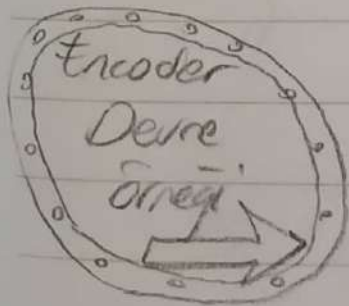
→ Decoding'in tersi. m -bit giriş kodunu $n \leq m \leq 2^n$ için n -bitlik çıkışa dönüştürür. (Decoding $\Rightarrow n \rightarrow m$ ($n \leq m \leq 2^n$))

→ Binary kodu Inputa girmesini sağlayan 2^n ya da daha az girişe sahiptir.

I_3	I_2	I_1	I_0	x	y	Ready
0	0	0	0	0	0	0
0	0	0	1	0	0	1
0	0	1	0	0	1	1
0	1	0	0	1	0	1
1	0	0	0	1	1	1
[GİRİŞLER]				[ÇIKIŞLAR]		



~Encoder Şeması~

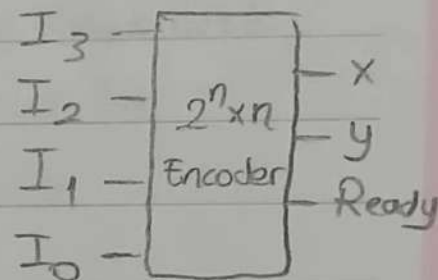


Encoding (Encoder: Kodlayıcı)

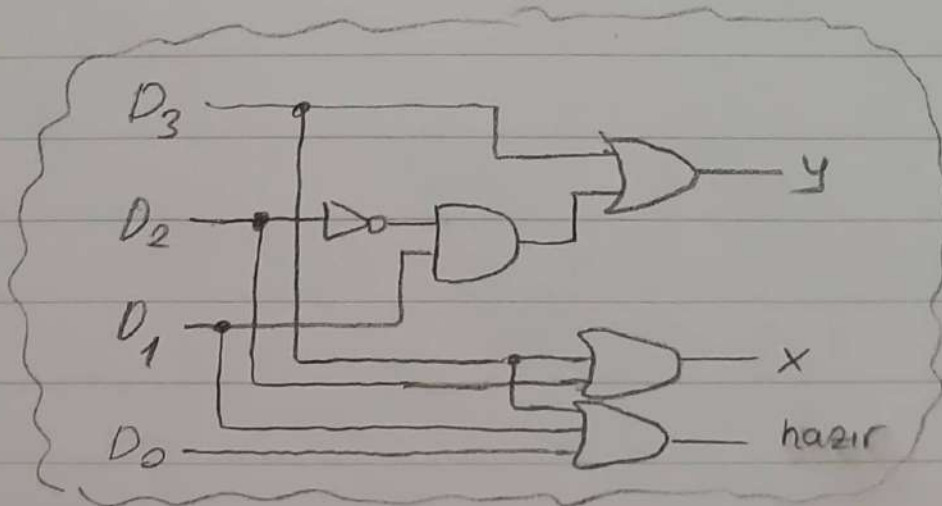
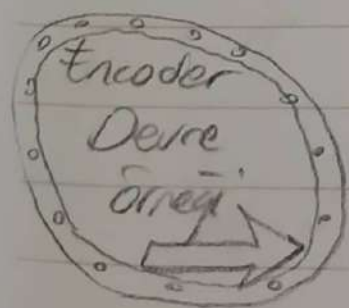
→ Decoding'ın tersi. m -bit giriş kodunu $n \leq m \leq 2^n$ için n -bitlik çıkışa dönüştürür. (Decoding $\Rightarrow n \rightarrow m$ ($n \leq m \leq 2^n$))

→ Binary kodu Inputa girmesini sağlayan 2^n ya da daha az girişe sahiptir.

I_3	I_2	I_1	I_0	x	y	Ready
0	0	0	0	0	0	0
0	0	0	1	0	0	1
0	0	1	0	0	1	1
0	1	0	0	1	0	1
1	0	0	0	1	1	1
[GİRİŞLER]				[ÇIKIŞLAR]		



~Encoder Şeması~

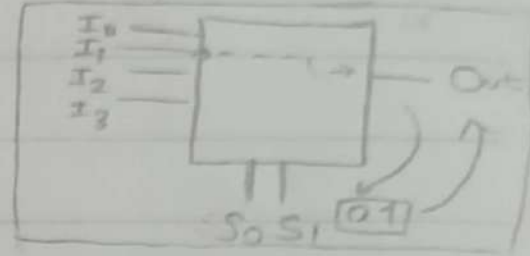


Multiplexer (MUX) (seçiciler)

→ Girişlerden birini çıkış olarak seçer.

→ Bunun için seçme verisini verecek uçlar gerekir.

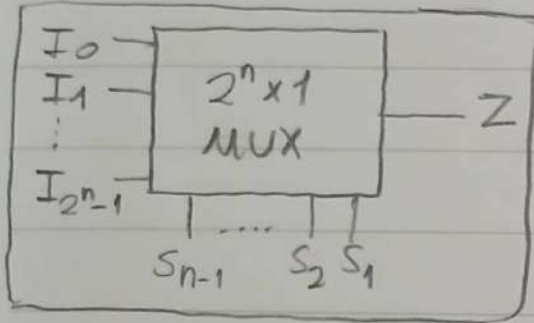
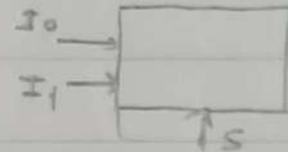
→ (2^n giriş için n uç girişi gerekir)



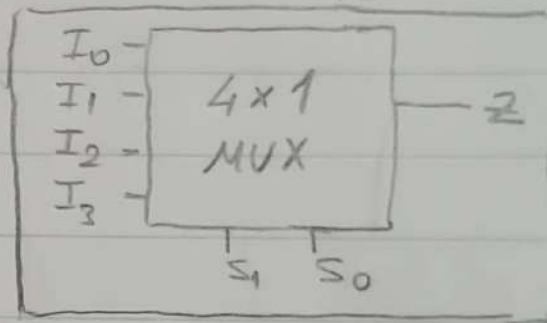
Örnek: 2-to-1 Line Multiplexer

$2 = 2^1 \rightarrow$ We need a single S (selection variable)

($S=0 \rightarrow$ selects input I_0)
($S=1 \rightarrow$ selects input I_1)

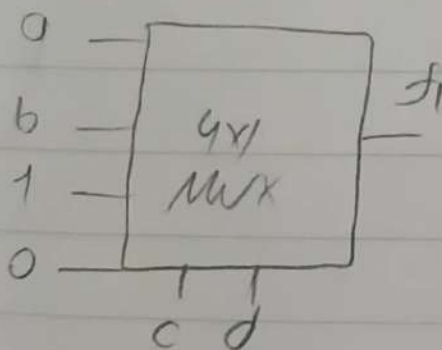


$2^n \times 1$
MUX



4×1
MUX

Mux ile doğru şey yapılabilir. Sınavda seçme ucunu seçtirerek veya direkt vererek doğruluk tablosu veya başka bir kombinezonsal devre gerçekleştirilmesi istenebilir.

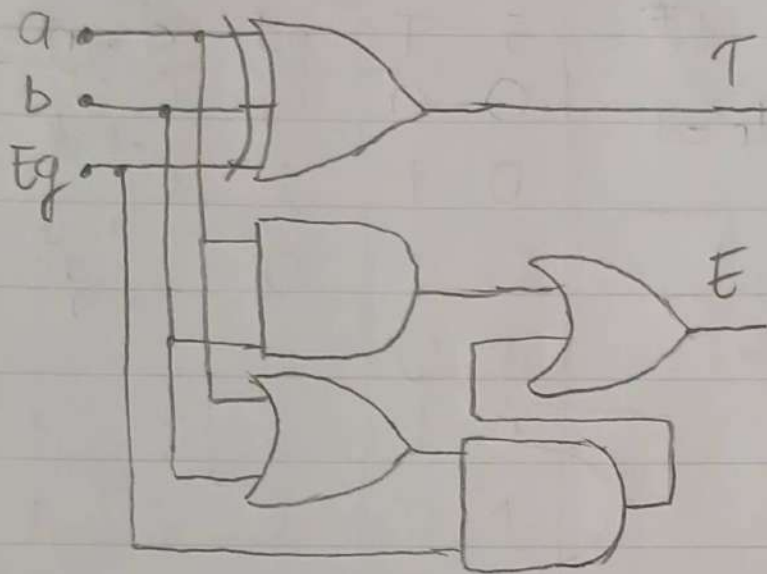


$$f_1 = a(c'd') + b(c'd) + 1(cd') + 0(cd)$$

Ödev² [Kombinezonsal Devreler]

a) Tam toplayıcıyı orijinal ve yarı toplayıcı halleriyle gerçekleyiniz.

Doğruluk tablosu kullanıp tasarımı elde edelim:



$$T = a \oplus b \oplus \bar{E}g$$

$$F = ab + Eg(a+b)$$

b) (Tam toplayıcı - yarı dizaynı arka sayfalarda)

c) 4 bitlik iki sayıyı toplayan devre

$$TT(a, b, C_{in}) \Rightarrow \begin{matrix} a_3 & a_2 & a_1 & a_0 \\ + & b_3 & b_2 & b_1 & b_0 \end{matrix} \quad (\text{digitale girildi})$$

d) Tam Çıkarıcı

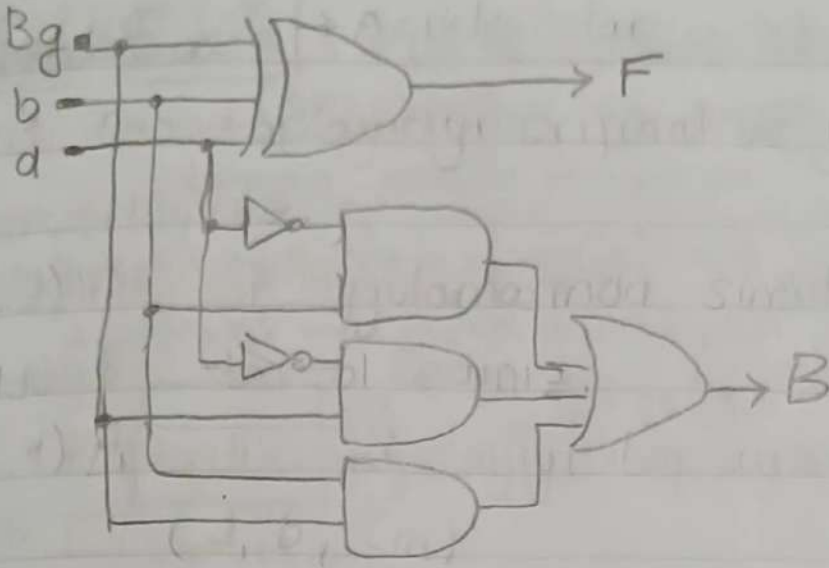
a	b	B _g	F	B
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

a \ b B _g	00	01	11	10
0	0	1	0	1
1	1	0	1	0

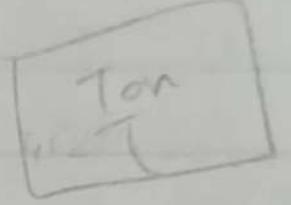
$$F = a \oplus b \oplus B_g$$

a \ b B _g	00	01	11	10
0	0	1	1	1
1	0	0	1	0

$$B = \bar{a}b + \bar{a}B_g + bB_g$$



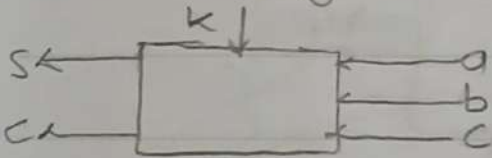
Tam
Gıkarıcı



- c.1) Tam gıkarıcıyı gerekleyiniz (K.map de kullan)
c.2) 4 bitlik gıkartma yapan devreyi iziniz. (Digital...?)
(ilk bor topraklanmah)

Ödev 3

4 giriş (a, b, e, k) ve 2 çıkışlı (c, s) devre, k singeli giriş
0 ise toplayıcı, 1 ise tam gıkartıcı olarak alışır.



- a) Doğruluk tablosunu,
b) ıkışların K.map ile bulunmasını,
c) TVE kullanarak semasını iziniz.

Ödev 4

irilen 4 bitlik A sayısını 2 arttıran devreyi iz.

B₀

→ ipucu

(zorunlu deėil)

Ödev 5

pdf 'deli' paragrafı K.Map ile tasarlayın (zorunlu deėil)

Ödev 6

→ Pdf Soru 4 YAP

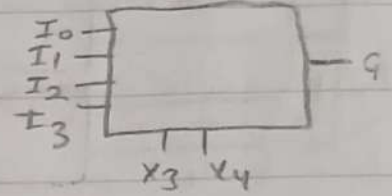
9. sayfa Digital

Aşağıda verilen doğruluk tablosunu 4×1 MUX kullanarak gerçekleştiriniz. (($x_3 - x_4$) seçme ucu kullanılacak)

x_1	x_2	x_3	x_4	f
0	0	0	0	0
0	0	0	1	0
0	0	1	0	1
0	0	1	1	1
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	1
1	0	1	0	1
1	0	1	1	0
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	0

⇒ Kanonik biçimde yazarsak:

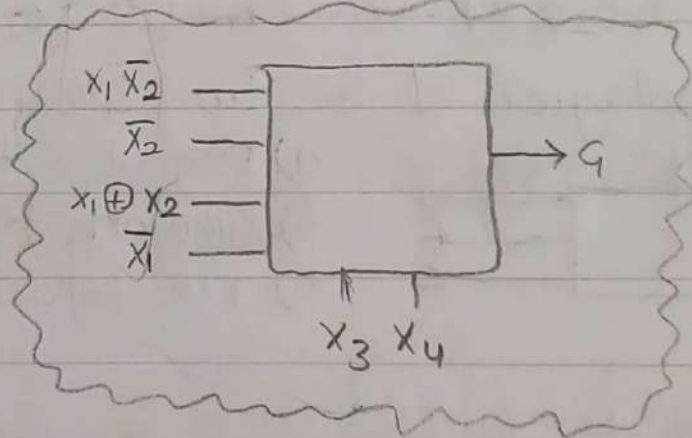
$$f = x_1'x_2'x_3x_4 + x_1'x_2'x_3'x_4 + x_1'x_2x_3'x_4 + x_1'x_2x_3x_4 + x_1x_2'x_3'x_4 + x_1x_2'x_3x_4 + x_1x_2x_3'x_4 + x_1x_2x_3x_4$$



$$f = x_3x_4'(x_1'x_2' + x_1x_2') + x_3x_4(x_1'x_2' + x_1'x_2) + x_3'x_4(x_1'x_2 + x_1x_2') + x_1x_2'x_3'x_4'$$

$$f = x_3x_4'(x_2') + x_3x_4(x_1')$$

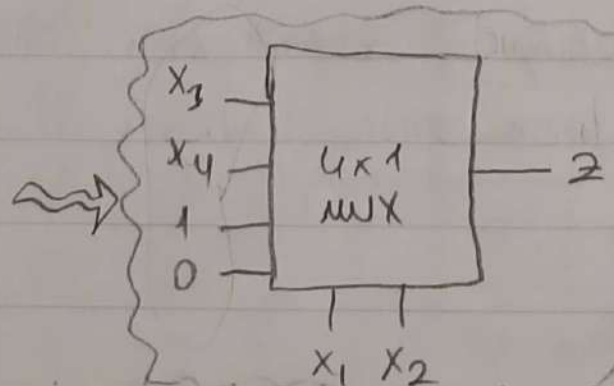
$$+ x_3'x_4(x_1 \oplus x_2) + x_3'x_4'(x_1x_2')$$



Aynı örneği ($x_1 - x_2$) seçme uçlarıyla yaparsak:

$$f = \bar{x}_1\bar{x}_2.(x_3) + \bar{x}_1x_2(x_4) + x_1.\bar{x}_2(1) + x_1x_2(0)$$

x_1	x_2	I
0	0	x_3
0	1	x_4
1	0	1
1	1	0



Çok daha başarılı!

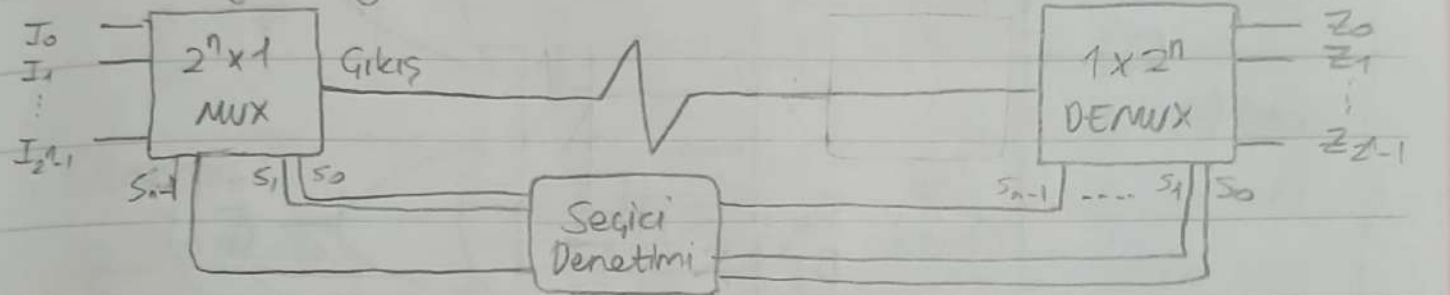
Ekstra kapi kullanımı ihtiyas duyulmadı.

⇒ Seçme girişlerinin belirlenmesi çok önemli.

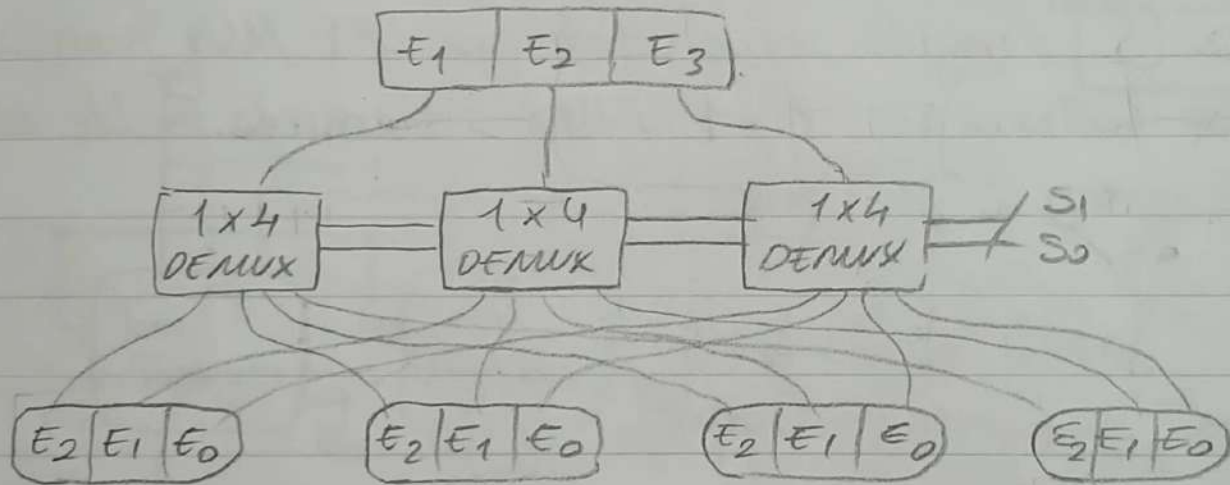
Demultiplexer (Dağıtıcı) (DEMUX)

→ Bir noktadan gelen giriş işaretini çıkış yolu seçip bu yola ileten kombinezonsal devre. (Aynı, n seçme vcu $\Rightarrow 2^n$ çıkış)

→ İlkel telefon santralleri MUX ve DEMUX devrelerini beraber kullanmaya bir örnektir. 2^n kullanıcıya böylece 2^n kullanıcı erişir.

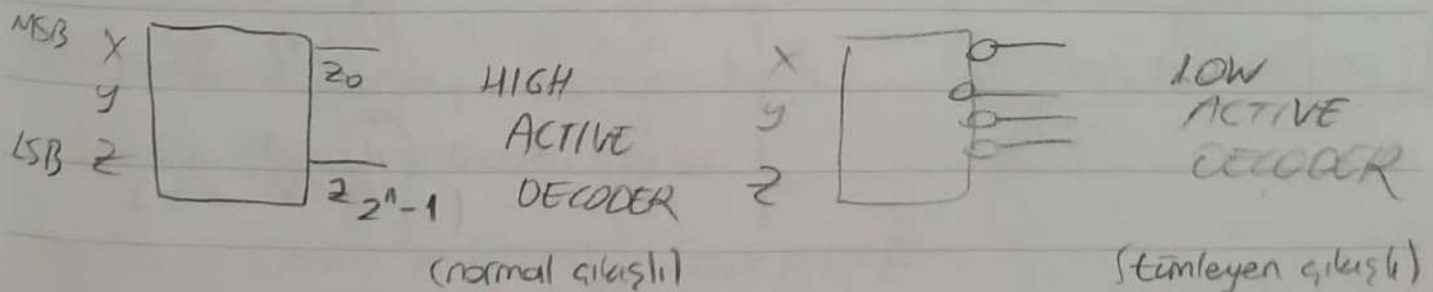


{ 3 bitlik bir ifadenin 4 kullanıcıya aktarılmasına örnek: }



[Türformasyon, seçim vclerına göre farklı kullanıcılar aktarma]

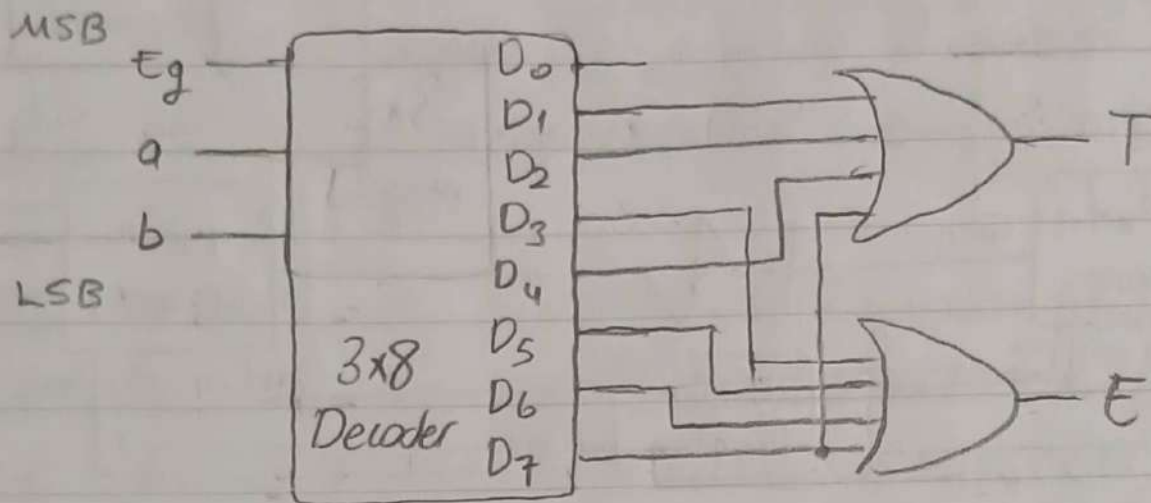
Sistemlere cihazlarına temel model olabilir?



{SORU: Tam Toplayıcıyı 3x8 Decoder'la nasıl gösteririz?}

$$T(Eg, a, b) = \sum(1, 2, 4, 7)$$

$$E(Eg, a, b) = \sum(3, 5, 6, 7)$$



Tab 3 Aşağıda verilen tabloyu 4x1 MUX'tan 2 tane kullanarak 8x1 MUX sistemiyle Felde ediniz.

A B C D F

↓ ↓ ↓ ↓

Enable Select input Output

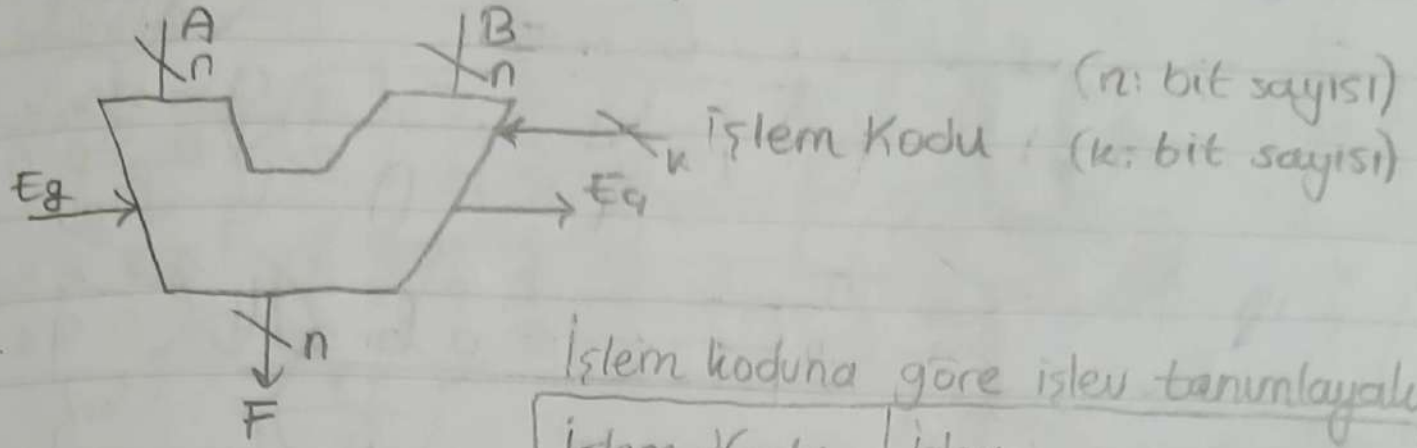
Mux 1

LAB

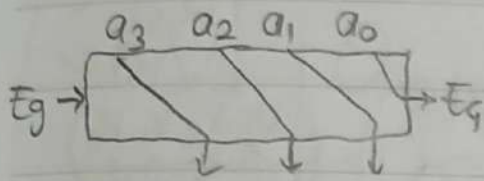
Mux 2

A	B	C	D	F		
1	0	0	0	1	>1	2C0
1	0	0	1	1	>1	2C0
1	0	1	0	0	>0	2C1
1	0	1	1	0	>0	2C1
1	1	0	0	0	>0	2C2
1	1	0	1	0	>0	2C2
1	1	1	0	1	>0'	2C3
1	1	1	1	0	>0'	2C3

ALU (Arithmetic Logic Unit)

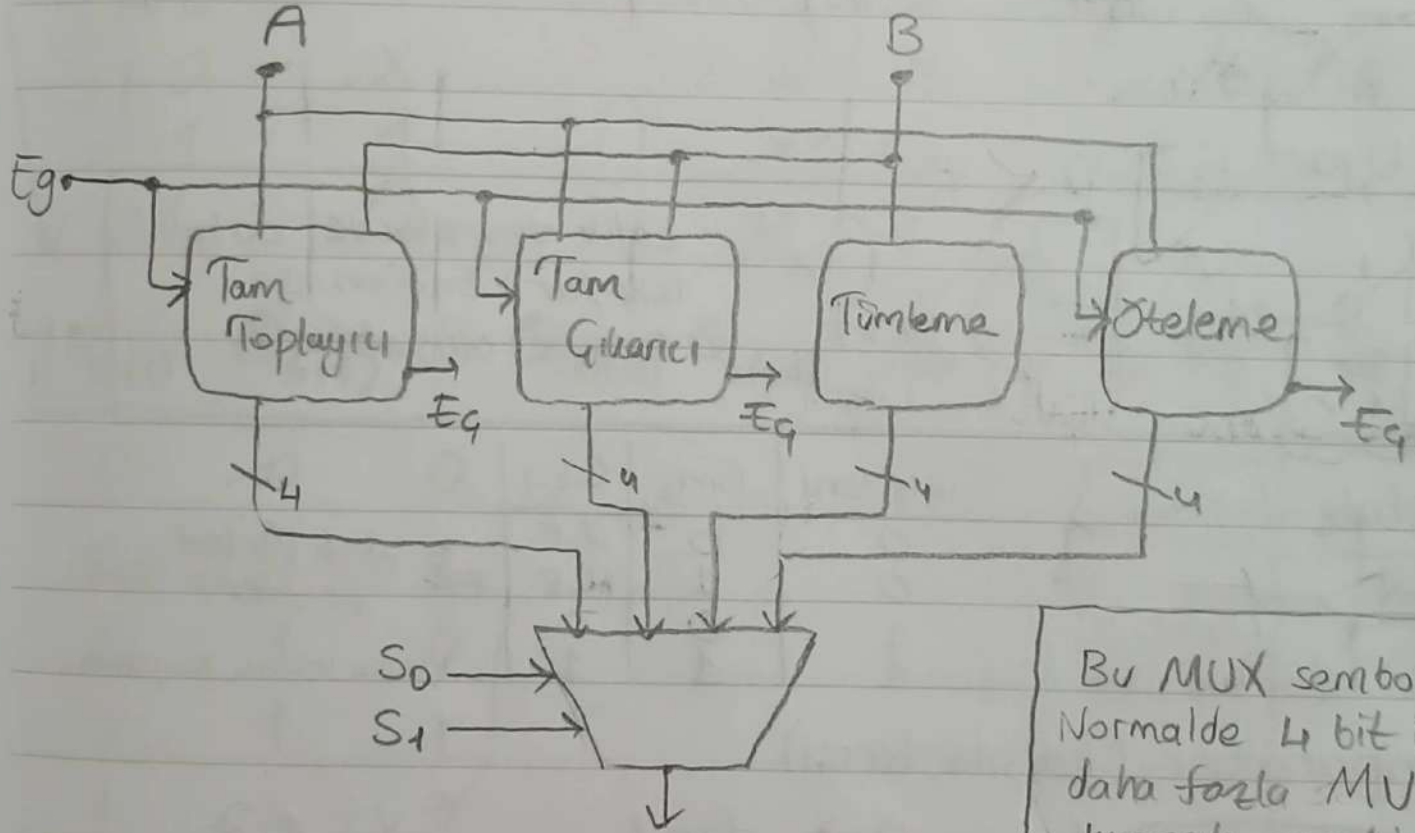


Öteleme?



İşlem koduna göre işlev tanımlayalım:

İşlem Kodu	İşlev
0 0	$F = A + B$ (4 bit)
0 1	$F = A - B$ (4 bit)
1 0	$F = \bar{B}$ (4 bit)
1 1	$F = A$ 'yı 1 bit sağa ötele.

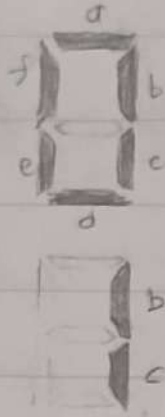
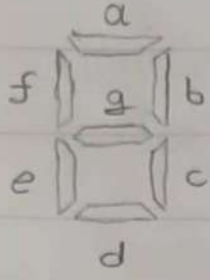


Bu MUX semboliktir. Normalde 4 bit için daha fazla MUX kurmak gerekir. (Eldeleri de bağlanı)

ALU işlemcilerde inanılmaz hızlı aritmetik işlem yapmaya yarar.

Register > Cache > RAM > Disk & USB

7 Segment - Display



Çıktılar						
a	b	c	d	e	f	g
1	1	1	1	1	1	0

Girdiler			
x_3	x_2	x_1	x_0
0	0	0	0

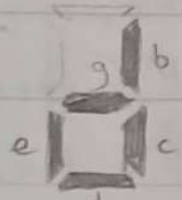
(0₁₆)

a	b	c	d	e	f	g
0	1	1	0	0	0	0

x_3	x_2	x_1	x_0
0	0	0	1

(1₁₆)

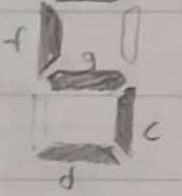
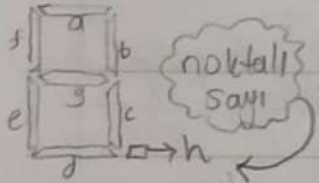
Hexadecimal (toplam 16) sayısal veriyi düşük maliyetle diyotlara yansıtır.



a	b	c	d	e	f	g
0	1	1	1	1	0	1

x_3	x_2	x_1	x_0
1	1	0	1

(D₁₆)



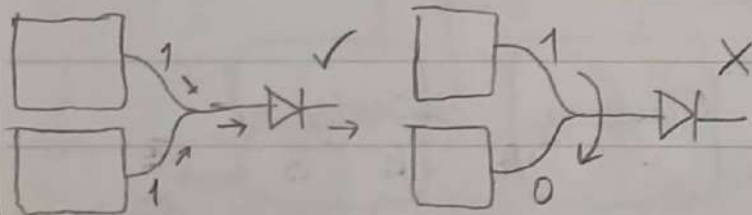
a	b	c	d	e	f	g
1	0	1	1	0	1	1

x_3	x_2	x_1	x_0
0	1	0	1

(5₁₆)

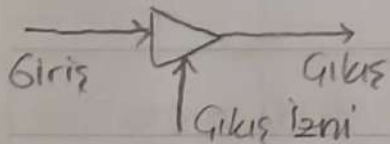


Uyarı



Lojik devrelerde output (çıkışlar) bağlanırsa sorunlar oluşabilir.

3-state-buffer

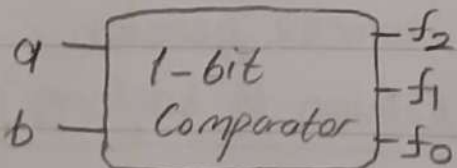


Çıkış izni	Giriş	Çıkış
0	0	Y.E
0	1	Y.E
1	0	0
1	1	1

⇒ High Active Buffer? (Bak)

(Y.E = Yüksek Empedans)

Comparator (Karşılaştırıcı)

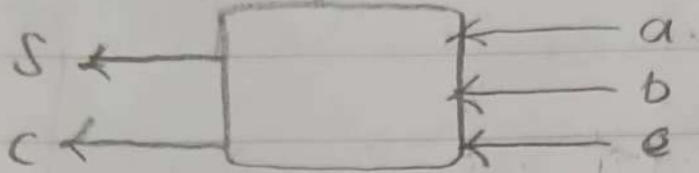


$a > b$	$f_0 = 1$
$a = b$	$f_1 = 1$
$a < b$	$f_2 = 1$

Y.E = 2

Ödev 4.

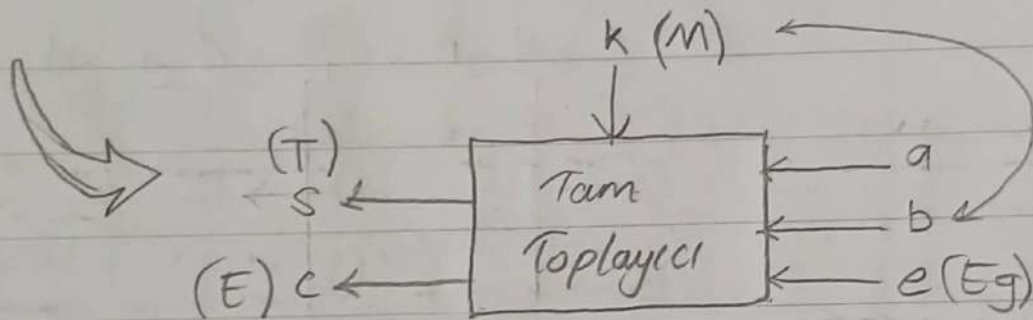
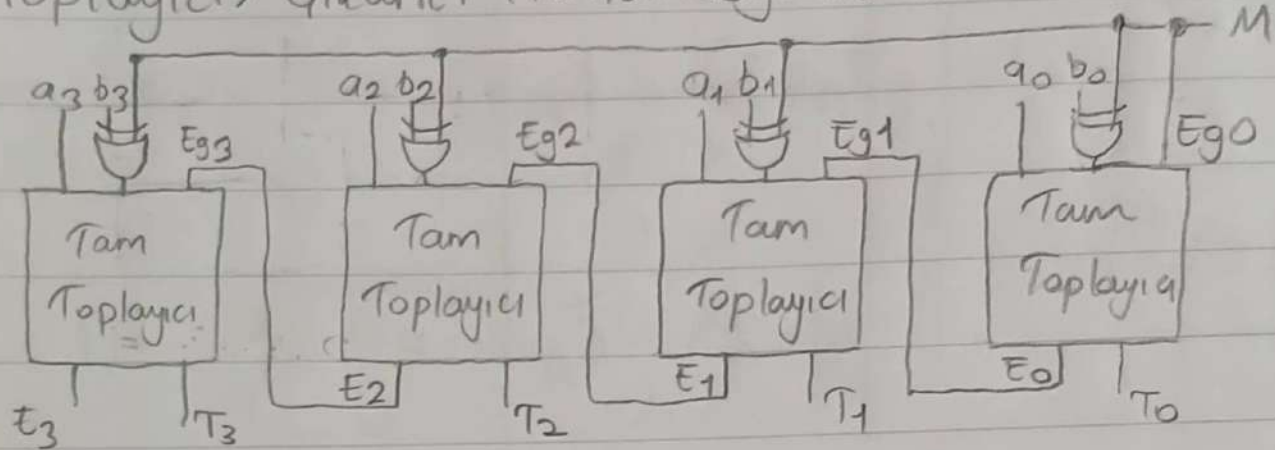
$\downarrow k$



k girişi 0 $\rightarrow$ TAM TOPLAYICI

k girişi 1 $\rightarrow$ TAM ÇIKARICI

Toplayıcı/Çıkartıcı Kombinasyonlu Devre



a) Doğruluk Tablosu

k	a	b	e	c	S
0	0	0	0	0	0
0	0	0	1	0	1
0	0	1	0	0	1
0	0	1	1	1	0
0	1	0	0	0	1
0	1	0	1	1	0
0	1	1	0	1	0
0	1	1	1	1	1
1	0	0	0	0	0
1	0	0	1	1	1
1	0	1	0	1	1
1	0	1	1	1	0
1	1	0	0	0	1
1	1	0	1	0	0
1	1	1	0	0	0
1	1	1	1	1	1

$\rightarrow$ k 'yı başa yazdık ki tablonun üstü toplama, altı çıkarma olsun.

a : birinci bir bitlik sayı

b : ikinci bir bitlik sayı

e : elde/borç

$k=0$ iken: $(a+b)+e=$

s (toplam), c (elde)

$k=1$ iken: $(a-b)-e=$

s (fark), c (borç)

b) Karnaugh Haritaları:

c (elde - borç):

be \ ka	00	01	11	10
00			1	
01		1	1	1
11			1	
10		1	1	1

$$C = be + \bar{k}ae + \bar{k}ab + k\bar{a}e + k\bar{a}b$$

S (toplam - fark):

be \ ka	00	01	11	10
00		1		1
01	1	1	1	
11	1		1	
10		1		1

$$S = a\bar{b}\bar{e} + \bar{a}\bar{b}e + abe + \bar{a}b\bar{e}$$

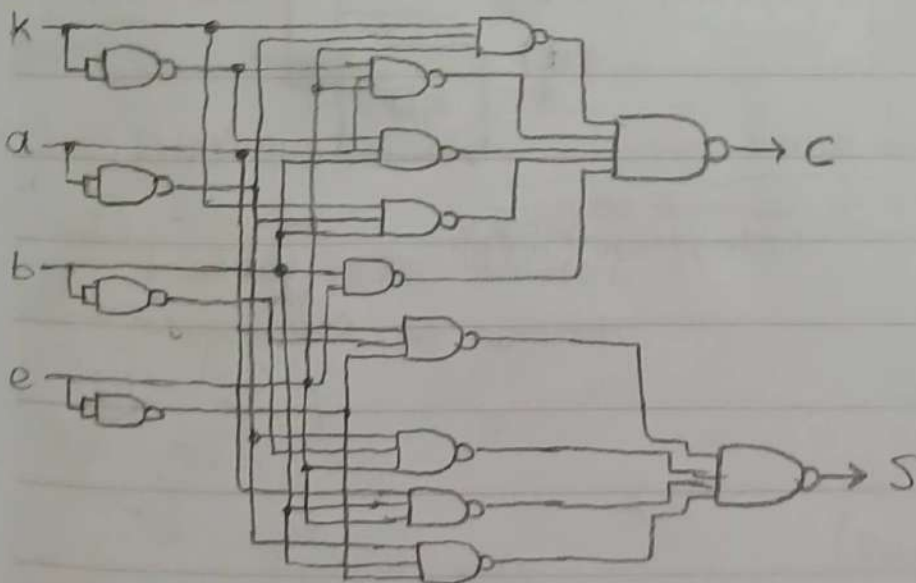
c) TVE kapıları kullanılarak çıkışları gerçekleştirme:

→ De Morgan kullanarak VEYA'ları VE'ye çevirelim:

$$\bar{C} = \overline{(be + \bar{k}ae + \bar{k}ab + k\bar{a}e + k\bar{a}b)} = ((\bar{b}\bar{e}) \cdot (\bar{k}\bar{a}e) \cdot (\bar{k}\bar{a}b) \cdot (k\bar{a}e) \cdot (k\bar{a}b))$$

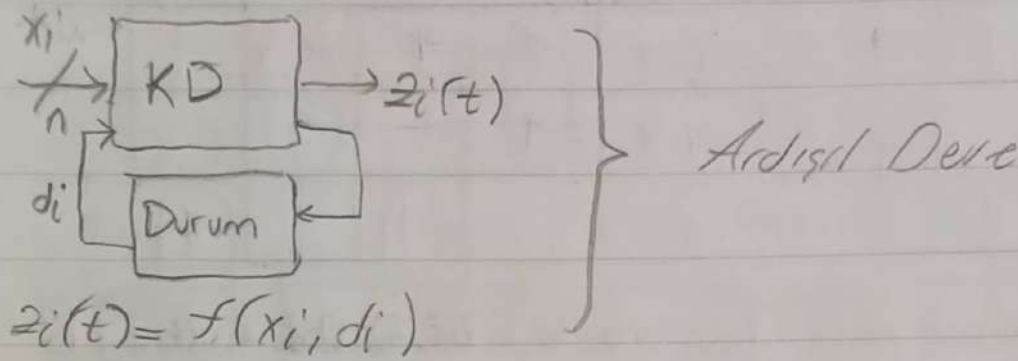
$$\bar{S} = \overline{(a\bar{b}\bar{e} + \bar{a}\bar{b}e + abe + \bar{a}b\bar{e})} = ((\bar{a}\bar{b}\bar{e}) \cdot (\bar{a}\bar{b}e) \cdot (abe) \cdot (\bar{a}b\bar{e}))$$

~ Devre Şeması ~



ARDIŞIL DEVRELER

→ Kombinezonsal devrelerin aksine hafıza kullanırlar.

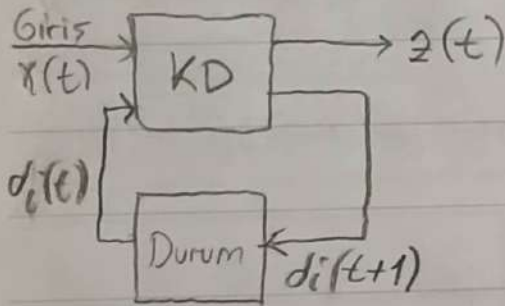


Ardışıl Devreler

sadece durumlardan yararlanır

Meally Makinesi → giriş ve durumdan yararlanır

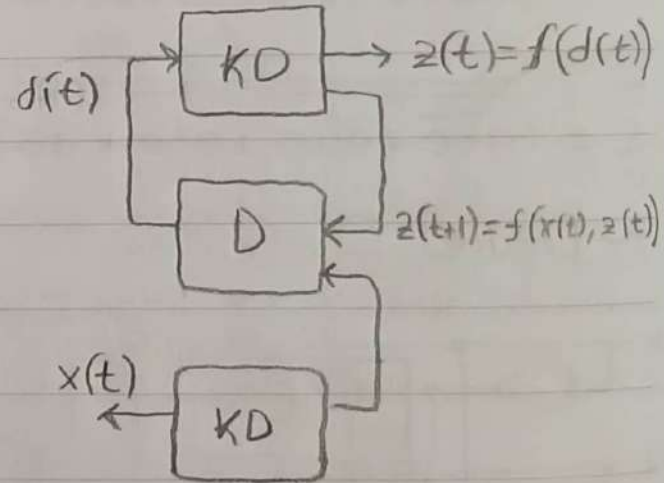
Moore Makinesi



$$z(t) = f(x(t), d(t))$$

$$d(t+1) = f(x(t), d(t))$$

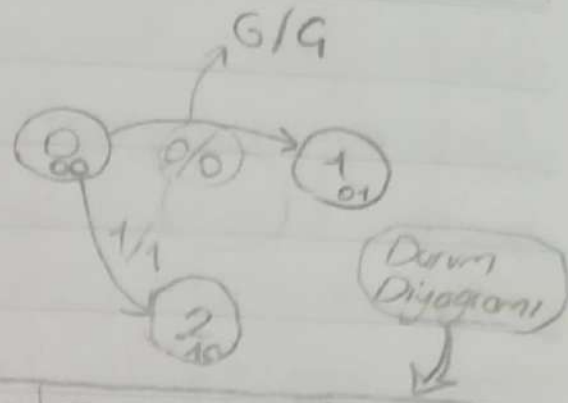
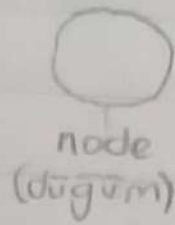
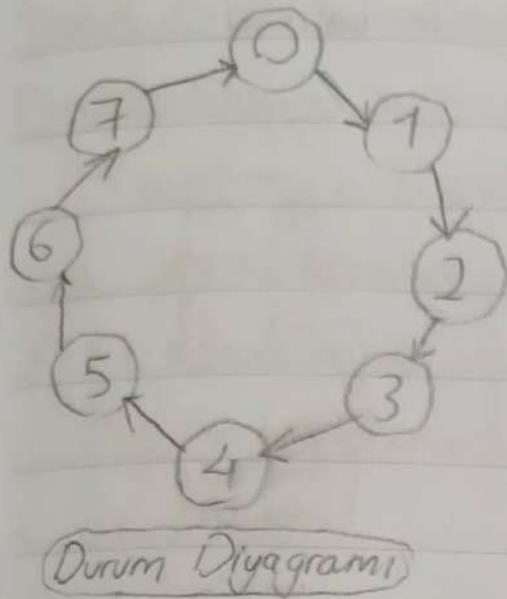
Çıktı durum ve girişin bir fonksiyonudur.



Sistemin çıktısı yalnızca durumun bir fonksiyonudur.

latch → mandal (saatsiz çalışır)

Durum ve Durum Diyagramları

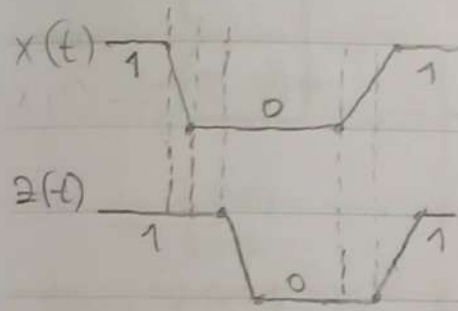


Şimdiki Durum	G	Sonraki Durum	Çıkış
$d_1 d_2$	x	$d_1^+ d_2^+$	z
0 0	0	0 1	0
0 0	1	1 0	1

Durum Tablosu

Asenkron & Senkron Ardışıl Devre

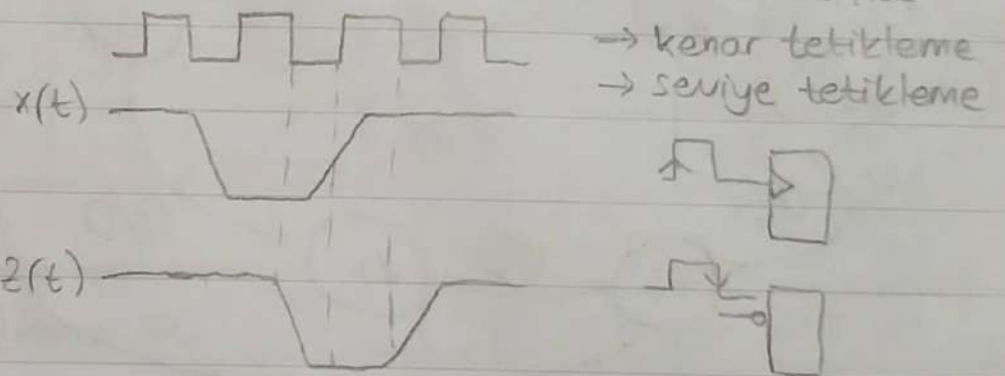
Asenkron → Gecikmelerden dolayı istenmeyen çıktıya dikkat edilmelidir.



(Gecikme Örneği)

Bu gecikme senkron devreler ile çözülür.

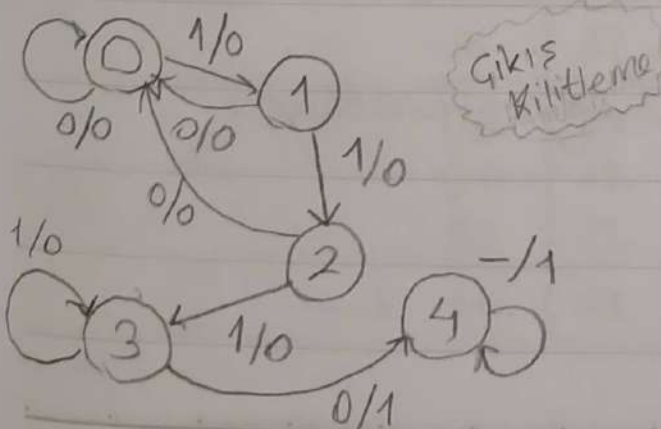
Senkron → saat işaretleri darbe başlık



örnek

Basit şifreleme devresi

$x(t)$ **1110** → $z(t)$ → Bu durumda 1 (şifre: 1110) aksi durumda 0



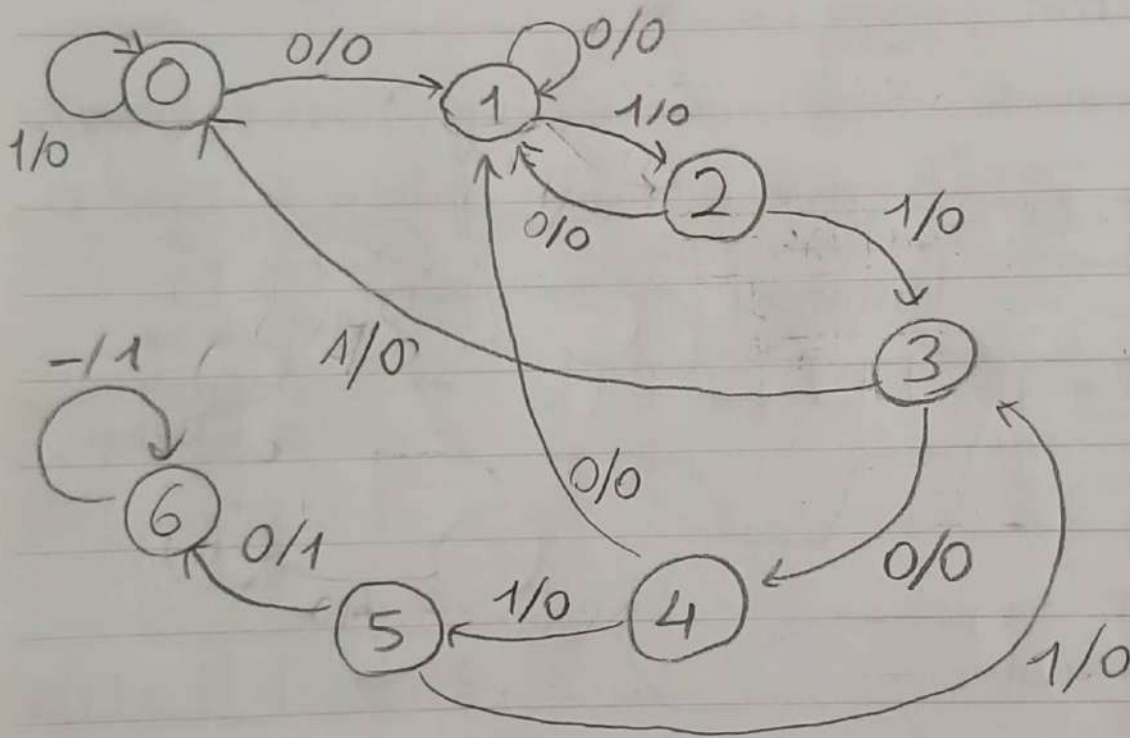
Sonlu Durum Makinaları (Finite State Machines)

Sonlu durum makinası çıkışı elde etmek için belirli sayıda geçmiş bilgiye ihtiyaç duyan bir ard. dev. uygulamasıdır.

Bazı ard. dev. uyg.'larında çıkış hemen o andaki durum bilgilerinden üretilirken bazı uygulamalarda çıkış belirli sayıda geçmişte kalan durum bilgilerine dayanılarak üretilir. (ör: şifre çözme devreleri)

örnek $\xrightarrow{t+}$

Şifre : 011010



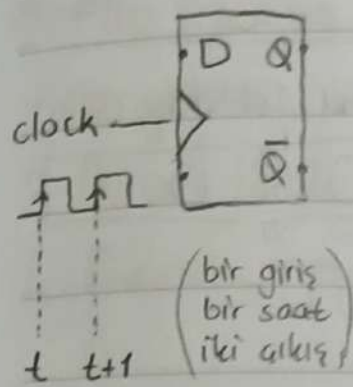
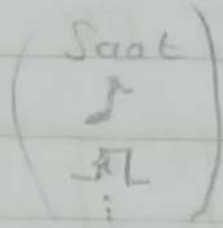
Ödev: En az 3 sayfa kombinezonsal devre mantığını anlatan özet yazın. (Programlanabilir - PLA-PAL vs.)

Ödev: Ana üyde FF ve tutucular 2 sayfa özetleyip sisteme yükleyin.

Flip-Floplar (İkідurumlular)

Temel 1 bitlik saklayıcılar.

D-Flip Flop (Delay)



$$Q(t+1) = D(t)$$

Giriş (D)	Saat	Çıkış (Q)
0	↓	0 0
0	↑	0 1
1	↓	1 0

Şimdiki çıkış: $Q(t): y$

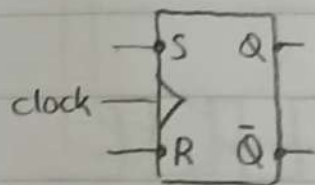
Sonraki çıkış: $Q(t+1): Y$

$Q(t) \backslash y$	00	01	11	10
$y \backslash D$	00	01	11	10
0	0	1	1	0

uyarı tablosu

$$Y = D$$

2-) SR Flip-Flop (Set-Reset)



2 giriş
saat
2 çıkış

$$Q(t+1) = Q(t) \cdot \overline{R(t)} + S(t)$$

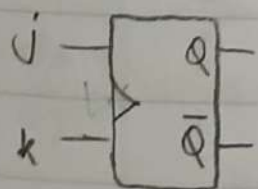
Giriş	Saat	Çıkış
S R	↓	$Q \overline{Q}$
00	↓	$Q \overline{Q}$
01	↓	0 1
10	↓	1 0
11	↓	Tanımsız.

$y \backslash SR$	00	01	11	10
0	0	0	∅	1
1	1	0	∅	1

$$Y = y \overline{R} + S$$

$y \backslash SR$	00	01	11	10
SR	0∅	10	∅0	01

3-) JK Flip-Flop (Gelişmiş SR)



SR ile farkı

$$Q(t+1) = Q(t) \cdot \overline{K(t)} + \overline{Q(t)} \cdot J(t)$$

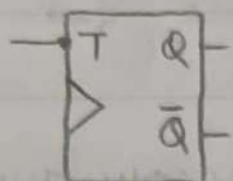
Giriş	Saat	Çıkış
JK	↓	$Q \overline{Q}$
00	↓	$Q \overline{Q}$
01	↓	0 1
10	↓	1 0
11	↓	$\overline{Q} Q$

$y \backslash JK$	00	01	11	10
0	0	0	1	1
1	1	0	0	1

$$Y = \overline{y} J + y \overline{K}$$

$y \backslash JK$	00	01	11	10
JK	0∅	1∅	∅0	∅1

4- T-FF (Toggle)



$$Q(t+1) = Q(t) \oplus T(t)$$

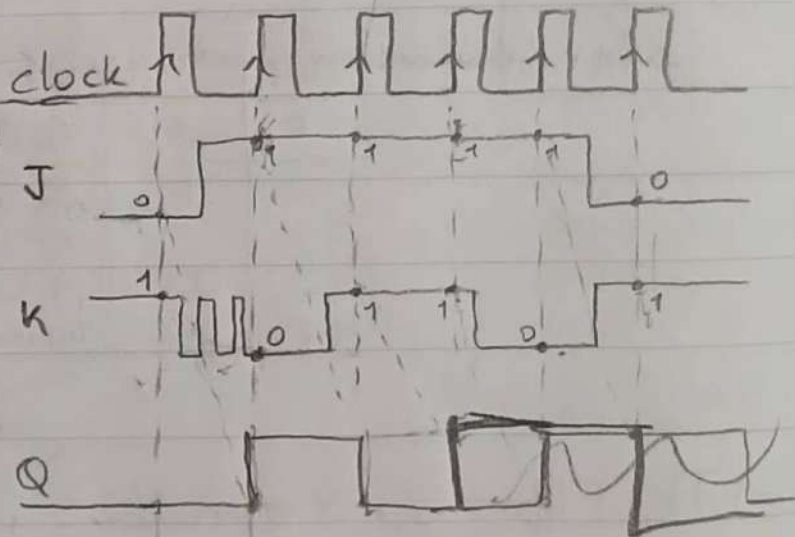
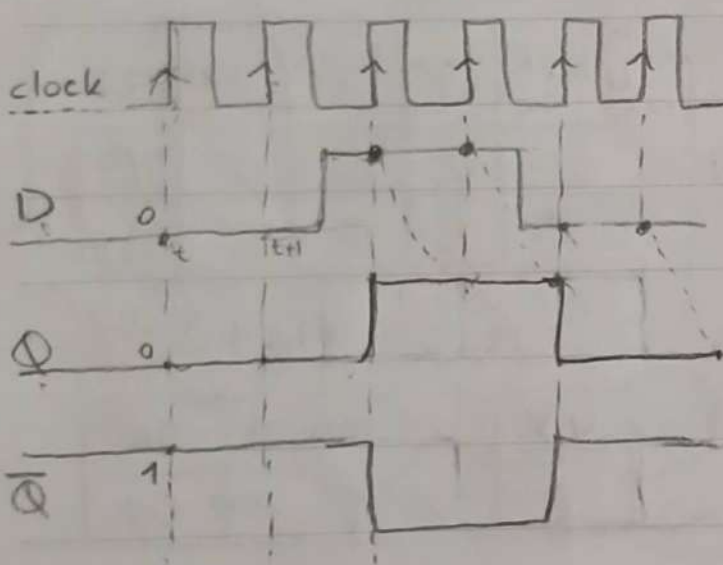
Giris	Saat	Gikis
T		Q Q-bar
0		Q Q-bar
1		Q Q-bar

y \ T	0	1
0	0	1
1	1	0

y \ T	00	01	11	10
T	0	1	0	1

$$Y = y\bar{T} + \bar{y}T = y \oplus T$$

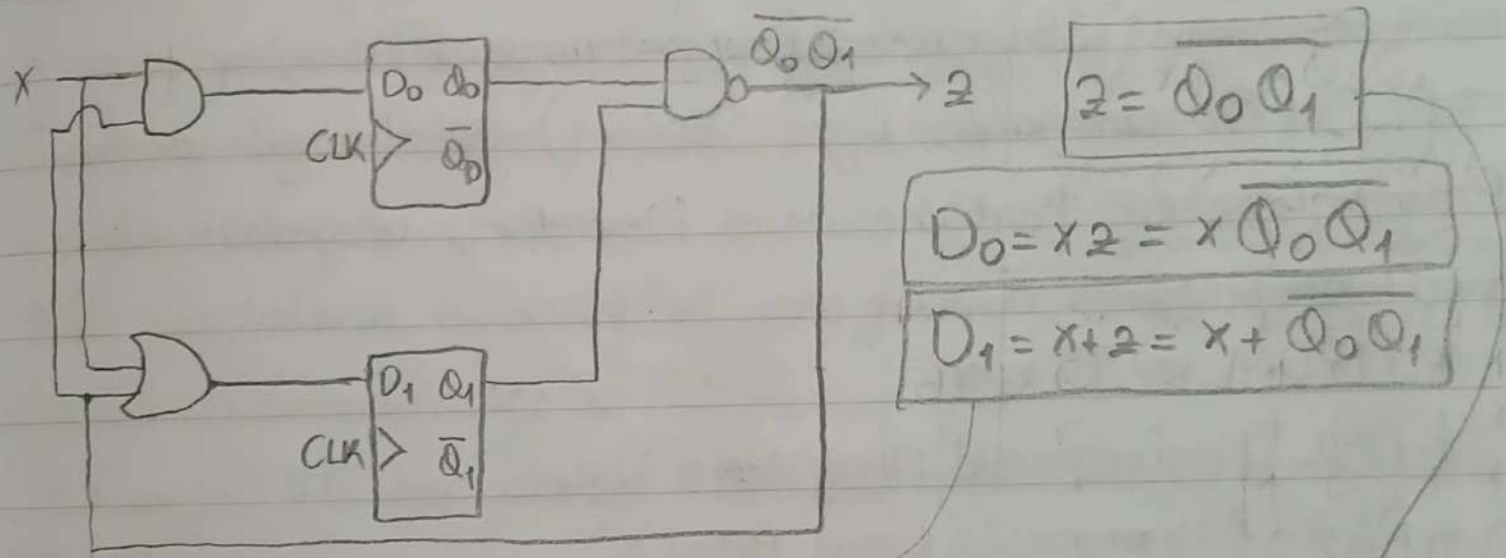
Elektriksel Sinyaller → Fiziksel Gıktılar



set

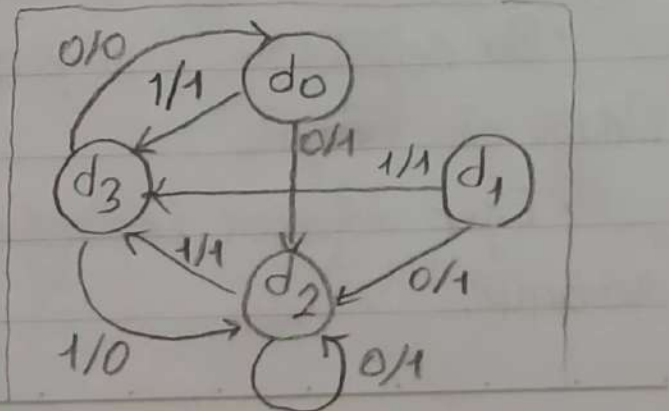
Ardışıl Devre Analiz Yöntemi

- 1-Flip Flopların girişlerine, ait ve devrenin çıkışlarına ilişkin Boole fonksiyonu devreye bakılarak yazılır.
- 2-Flip Flop'un geçiş fonksiyonları kullanılarak bir sonraki durum fonksiyonları elde edilir.
- 3-Durum tablosu oluşturulur.
- 4-Durum diyagramı çizilir.



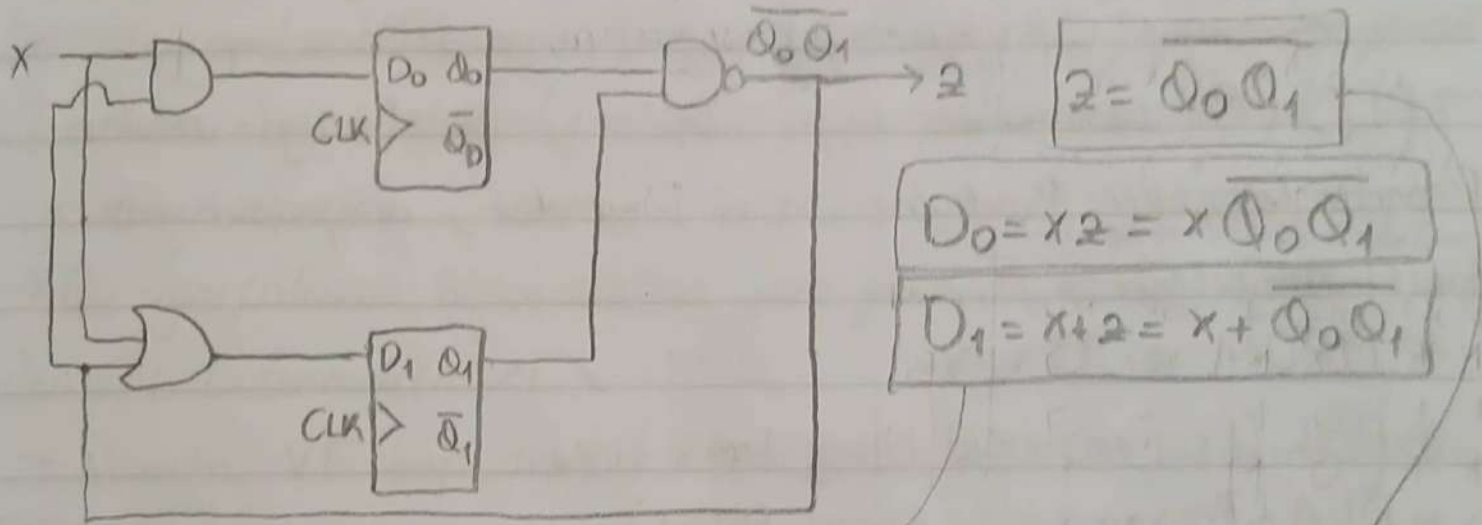
	t						t+1					
	Current State			Flip-Flop Girişleri				Next State				Çıkış
	Şimdiki Durum							Sonraki Durumlar				
Simgesel Durum	Q ₁	Q ₀	y	x=0		x=1		x=0		x=1		z
				D ₁	D ₀	D ₁	D ₀	Q ₁ ⁺	Q ₀ ⁺	Q ₁ ⁺	Q ₀ ⁺	
d ₀	0	0		1	0	1	1	1	0	1	1	1
d ₁	0	1		1	0	1	1	1	0	1	1	1
d ₂	1	0		1	0	1	1	1	0	1	1	1
d ₃	1	1		0	0	1	0	0	0	1	0	0

Şimdiki Durum	Bir Sonraki Durumlar		Çıkış
	$x=0$	$x=1$	z
d_0 (00)	d_2	d_3	1
d_1 (01)	d_2	d_3	1
d_2 (10)	d_2	d_3	1
d_3 (11)	d_0	d_2	0



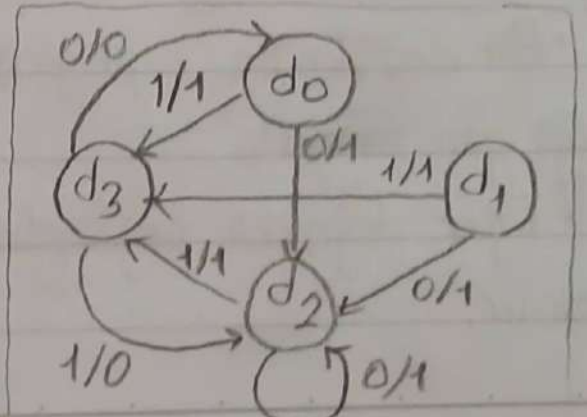
Ardışıl Devre Analiz Yöntemi

- 1-Flip Flopların girişlerine, çıkış ve durumun çıkışlarına ilişkin Boole fonksiyonu devreye bakılarak yazılır.
- 2-Flip Flop'un geçiş fonksiyonları kullanılarak bir sonraki durum fonksiyonları elde edilir.
- 3-Durum tablosu oluşturulur.
- 4-Durum diyagramı çizilir.



Simgesel Durum	t								t+1				Çıkış
	Current State Şimdiki Durum			Flip-Flop Girişleri				Next State Sonraki Durumlar					
	Q ₁	Q ₀	y	x=0		x=1		x=0		x=1			
				D ₁	D ₀	D ₁	D ₀	Q ₁ ⁺	Q ₀ ⁺	Q ₁ ⁺	Q ₀ ⁺	Σ	
d ₀	0	0		1	0	1	1	1	0	1	1	1	
d ₁	0	1		1	0	1	1	1	0	1	1	1	
d ₂	1	0		1	0	1	1	1	0	1	1	1	
d ₃	1	1		0	0	1	0	0	0	1	0	0	

Şimdiki Durum	Bir Sonraki Durumlar		Çıkış
	x=0	x=1	z
d ₀ (00)	d ₂	d ₃	1
d ₁ (01)	d ₂	d ₃	1
d ₂ (10)	d ₂	d ₃	1
d ₃ (11)	d ₀	d ₂	0



Programlanabilir Kombinezonsal Devreler (PLD)

Fiziksel olarak tek bir tümdevre içerisinde birçok VE-VEYA kapıları bulunan ve karmaşık lojik ifadelerin bağlantıları uygun bir şekilde programlanabilen tek bir tümdevre ile gerçekleştirilmesini sağlayan yarı hazır elemanlar "Programlanabilir Kombinezonsal Devreler" olarak adlandırılır.

→ Lojik ifadelerin karmaşıklığı arttıkça kapı tümdevreleri yerine orta ölçekli tümdevreler ya da PLD kullanma yaklaşımı, kart tasarımı yalınlaştırdığından daha yararlıdır.

→ PLD (Programmable Logic Devices) genel adıyla anılan Programlanabilir Kombinezonsal Devreler, aşağıdaki gibi sınıflanmaktadır:

- * PROM ve Türeveleri

- * PAL (Programmable Array Logic)

- * PLA (Programmable Logic Array)

- * PSA (Programmable Sequential Array)

Programlanabilir olma özelliği, lojik kapılar bağlamında istenilen işlevin sağlanması anlamına gelir.

→ Bu programlama işi kapı girişlerine bağlantı sağlayan sigortalar ile sağlanır:

- Bir değişkenin kapı girişine etkimesi isteniyorsa sigorta olduğu gibi bırakılır.

- Eğer değişken bağlantısı istenmiyorsa ilgili sigortta arttırılır.

→ Bazı PLD elemanlarını tekrar programlamak mümkünken bazı elemanlarda bu mümkün değildir.

ÖDEV

Bu yöntemleri ile iki sınıfa ayrılırlar:

- Bir kez programlanabilen: PROM, PAL ve PLA
 - Defalarca programlanabilen: EPROM, E²PROM ve LCA/LGA
- (Erasable Programmable Read Only Memory) (Electrically EPROM) (Logic Cell Arrays / Logic Gate Arrays)

PLD Elemanları

Bir programlanabilir kombinezonsal devre, VE kapı dizisi, VEYA kapı dizisi ve bunların programlanmasına imkan veren sigortalar içerir.

→ Bu sigortaların bulundukları yere göre üç farklı PLD elemanı bulunmaktadır:

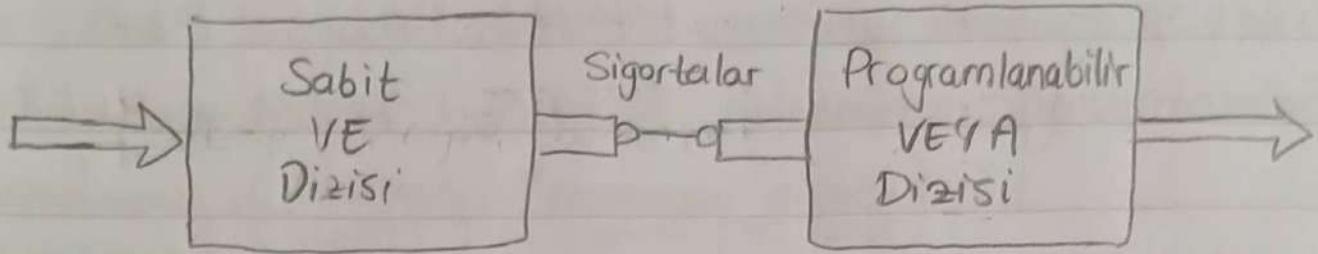
- Sigorta, VE kapı dizisi ile VEYA kapı dizisi arasındaysa: PROM
- Sigorta, giriş işareti ile VE kapı dizisi arasındaysa: PAL
- Hem giriş - VE dizisi arasında hem de VE - VEYA kapı dizileri arasındaysa: PLA

→ Sigortanın konumu neyin programlanabileceğini belirler. Önünde bulunduğu kapının programlanabilmesi anlamına gelir.

Şimdi bu 3 PLD elemanını inceleyelim:

PROM (Programmable Read Only Memory):

PROM, sabit VE kapı dizisi ve sigortalarla programlanabilen VEYA kapı dizisi şeklinde bir mimariye sahip tümleşik elemandır.



(Sadece VEYA dizisi programlanabilir)

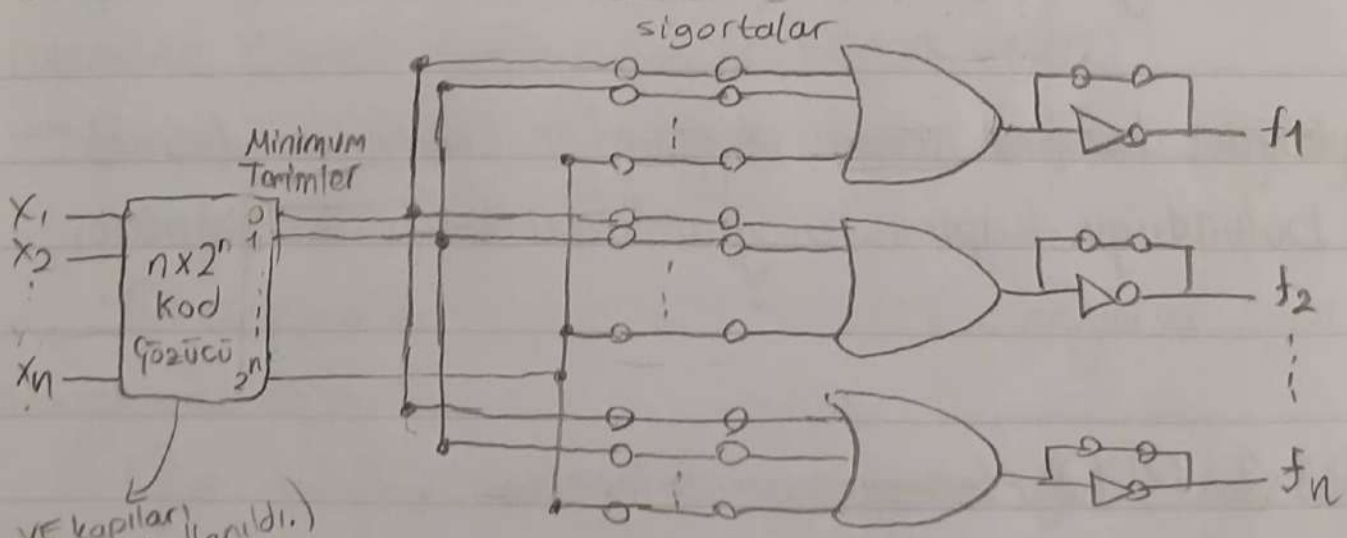
Programlama, bağlı olmayacak VE ve VEYA kapı bağlantılarını sağlayan sigortaların attırılmasıyla olur.

→ PROM, minimum terimler kanonik biçiminde verilmiş olan lojik fonksiyonların gerçekleştirilmesi için uygundur.

PROM'un İç Yapısı

PROM devresi n giriş, m çıkışlı kombinezonsal bir devre mimarisidir. $2^n \times m$ bitlik PROM olarak isimlendirilir.

PROM bir bellek türüdür, ROM ailesinin bir üyesidir. Bilgisayar ve sayısal sistemler üzerinde kalıcı verilerin veya program kodlarının tutulması/saklanması için kullanılır.



(PROM elemanının mimarisi,)

PROM'a ilişkin bir örnek:

ÖDEV

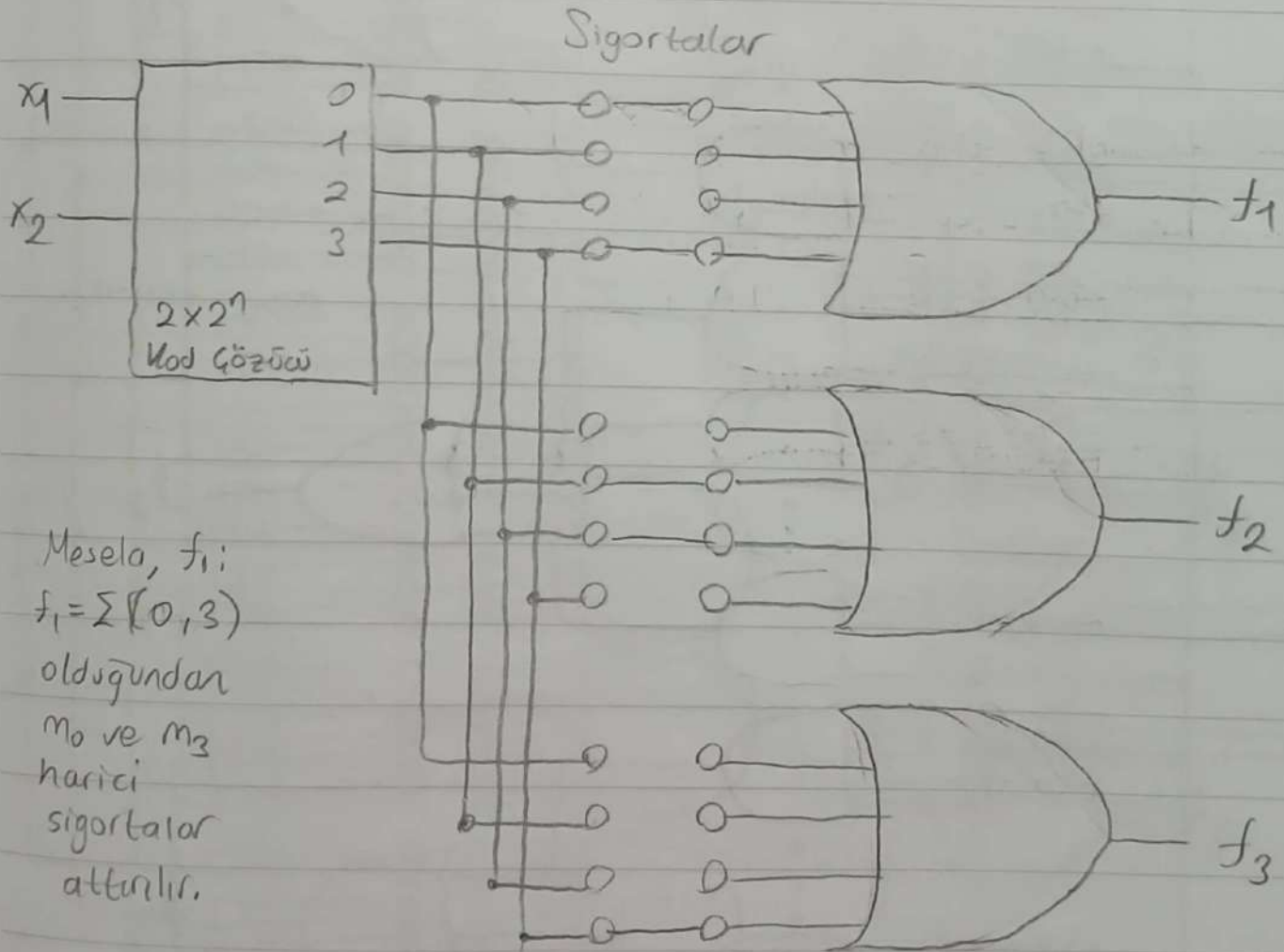
x_1	x_2	f_1	f_2	f_3
0	0	1	0	0
0	1	0	1	0
1	0	0	1	0
1	1	1	0	1

Lojik ifadesi yandaki gibi verilen fonksiyonları PROM elemanı ile gerçekleştiriniz.
→ Kaç bitlik PROM gerekir?

→ Devre 2 girişli, 3 çıkışlıdır; bu durumda $2^2 \times 3$ bitlik PROM elemanına ihtiyaç vardır.

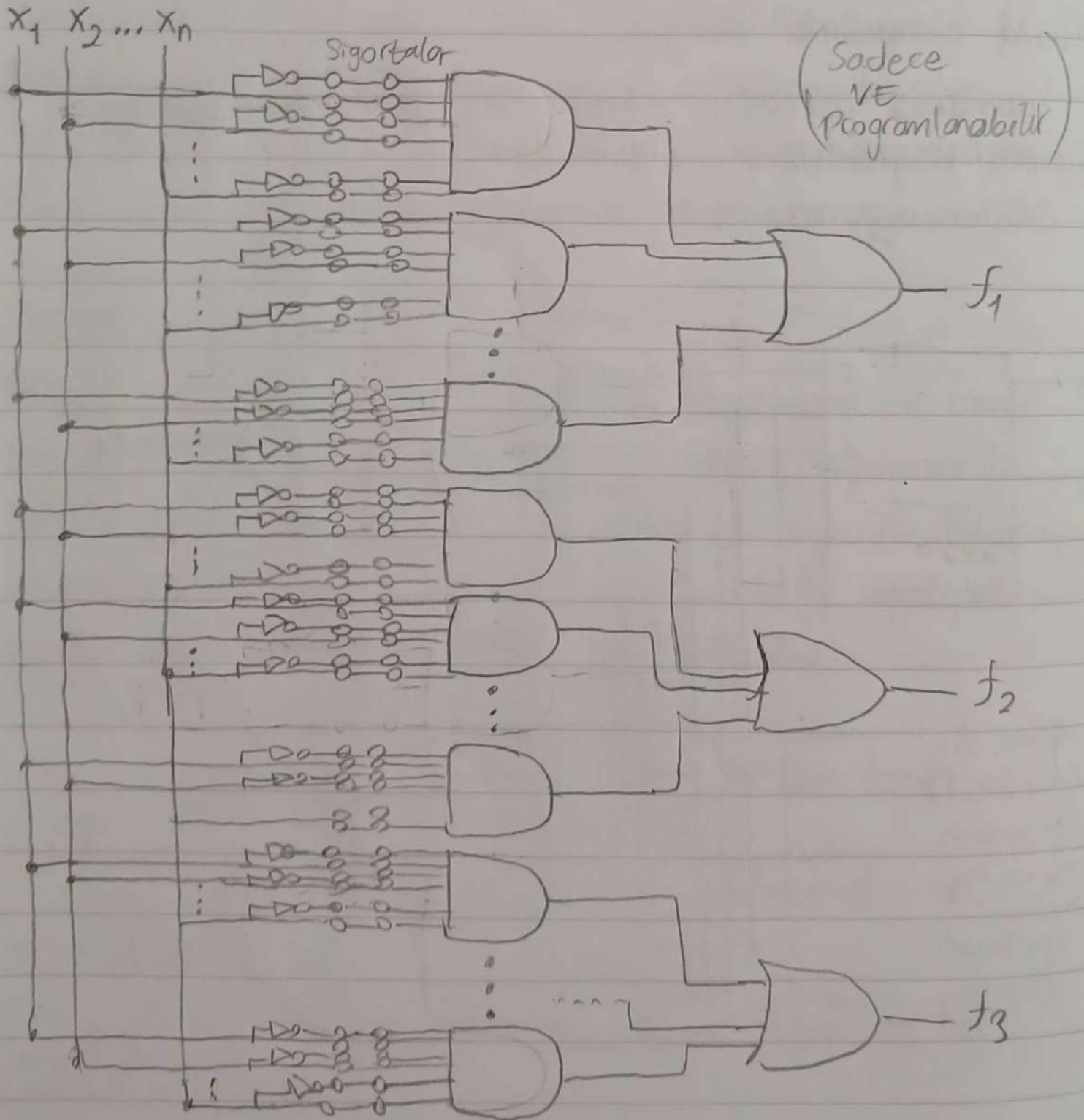
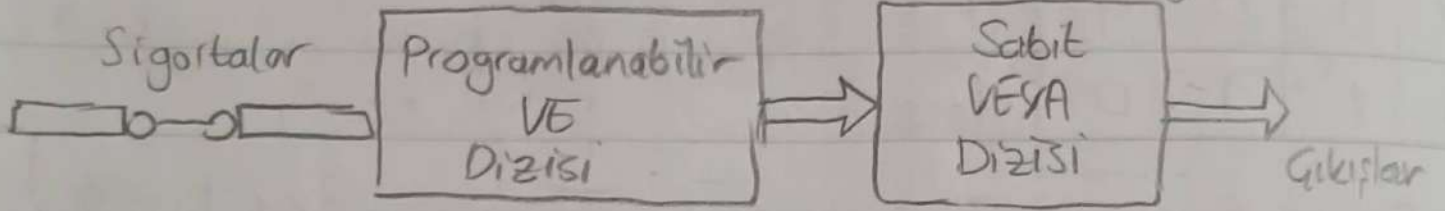
Yapmamız gereken minterimlere göre istenmeyen sigorta bağlantılarını attırmaaktır.

Oluşturacağımız devre şu şekilde olur:



PAL (Programmable Array Logic / Programlanabilir Dizi)

Sadece VE kapılarının programlanabildiği PLD'ye kısaca PAL denir. PAL elemanıya aynı şekilde min. terimler kanonik biçimde verilmiş ifadeler gerçekleştirir.



PAL 1g Yapısı

PAL için bir örnek:

		f_1						f_2						f_3			
		cd						cd						cd			
ab		00	01	11	10	ab		00	01	11	10	ab		00	01	11	10
00			1		1	00		1	1	1	1	00		1			1
01			1			01						01			1		
11		1	1			11				1	1	11			1		
10		1	1			10		1	1	1	1	10		1			1

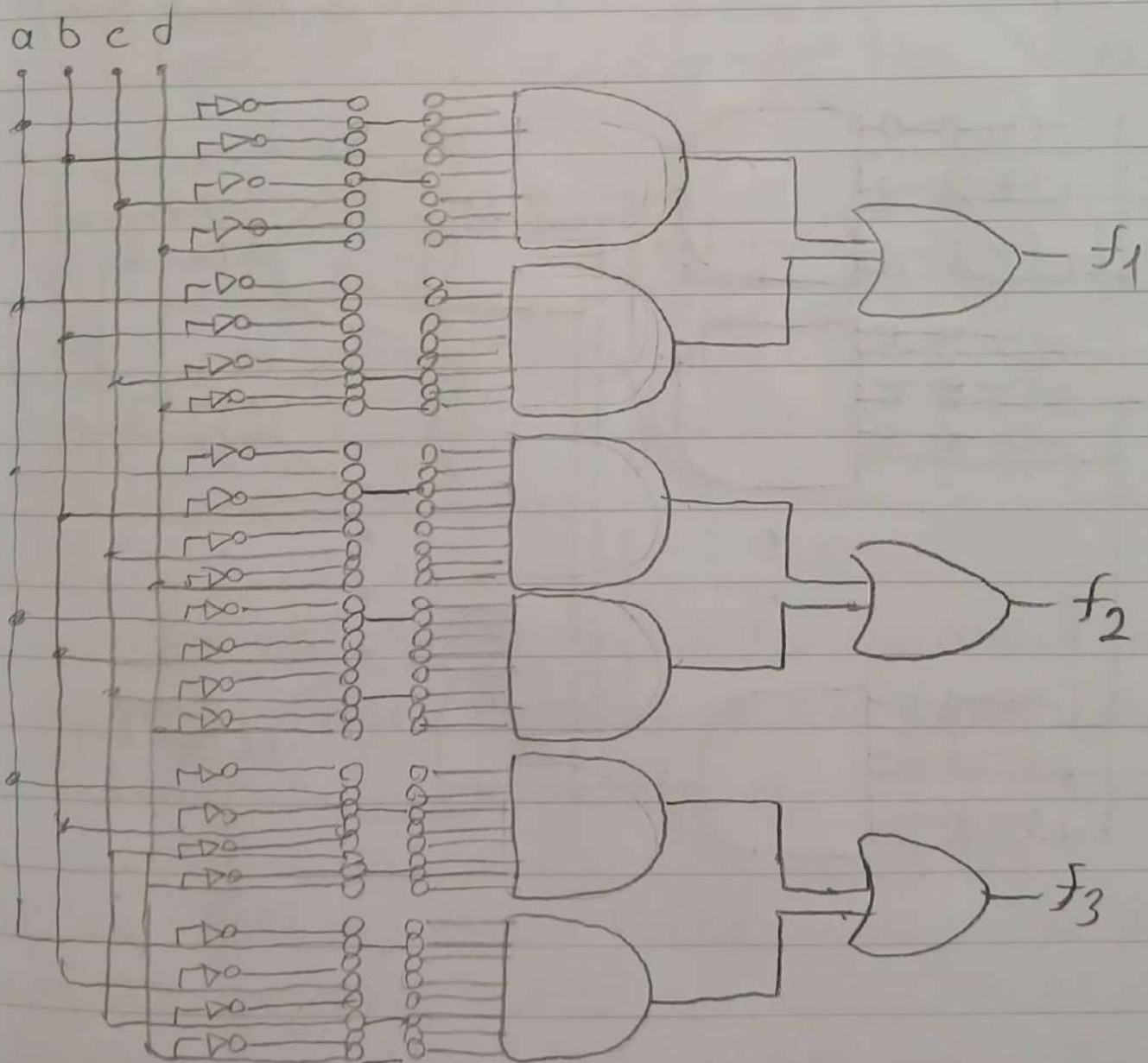
Bu fonksiyonları PAL ile ifade etmek için önce indirgeyelim.

$$f_1 = a\bar{c} + cd$$

$$f_2 = \bar{b} + ac$$

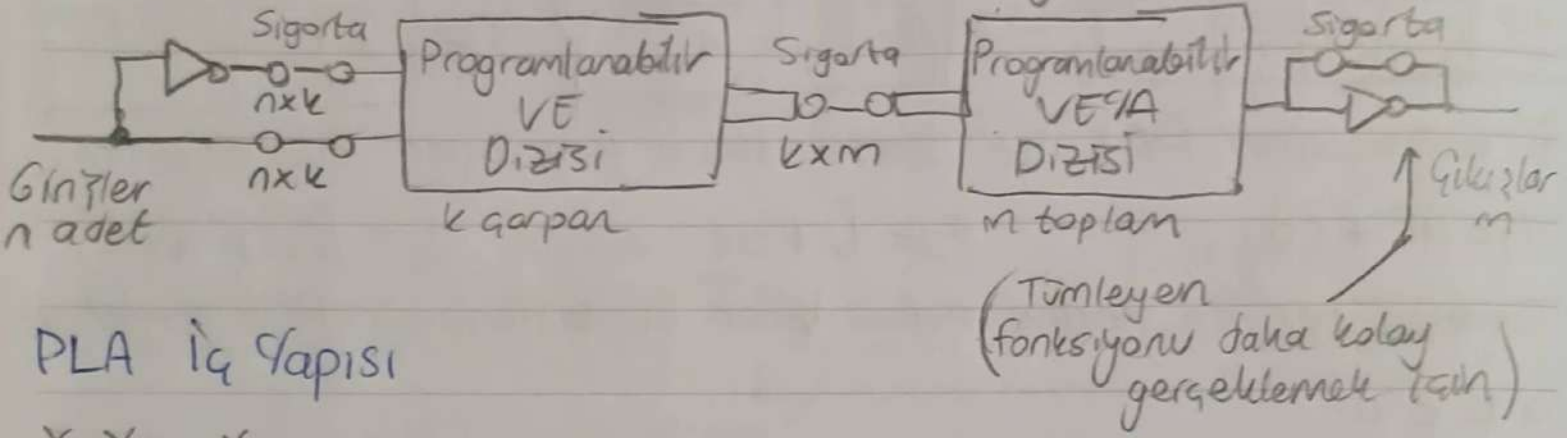
$$f_3 = \bar{b}\bar{d} + acd$$

Toplanan her bir terim için birer VE kullanarak PAL düzeldim.



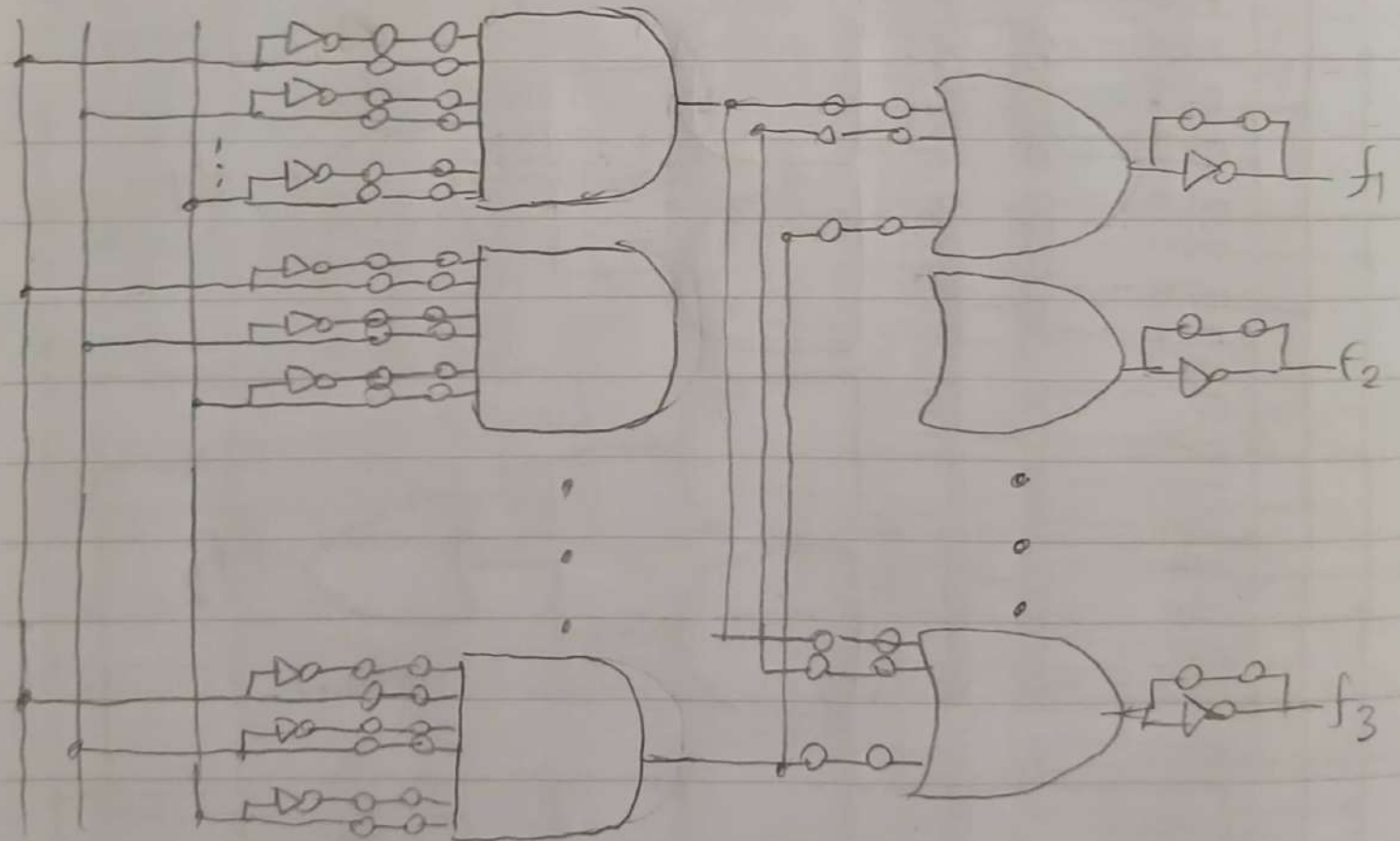
PLA (Programmable Logic Array)

PROM ve PAL programlaması tek bir dizide sigorta kullanıldığından basittir, ancak PLA ise iki noktada da sigorta kontrolü yaparak daha esnek çözümler sunar.
→ Hem VE hem VEYA kapıları programlanabilir.



PLA İç Yapısı

$x_1 x_2 \dots x_n$



PLA için örnek:

$$f_1 = x_2 x_3 + x_2 \bar{x}_4 + \bar{x}_2 \bar{x}_3 x_4$$

$$f_2 = \bar{x}_2 + \bar{x}_3 x_4$$

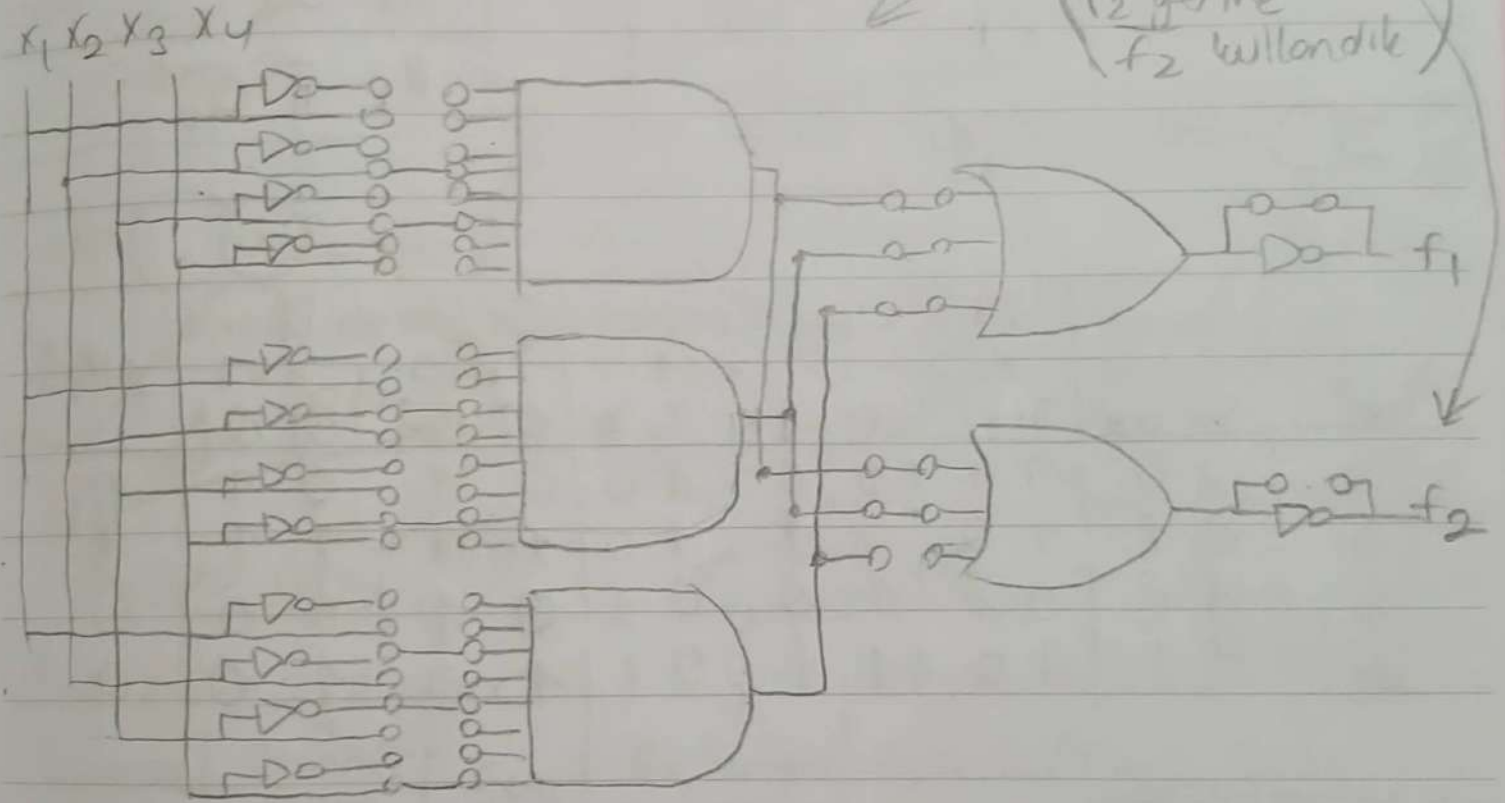
$$f_2 = x_2 x_3 + x_2 \bar{x}_4$$

Bu iki fonksiyonu PLA ile gerçekleştirelim.

Görüldüğü gibi bazı ortak terimler var.

O zaman daha az kapı kullanarak maliyeti düştebiliriz.

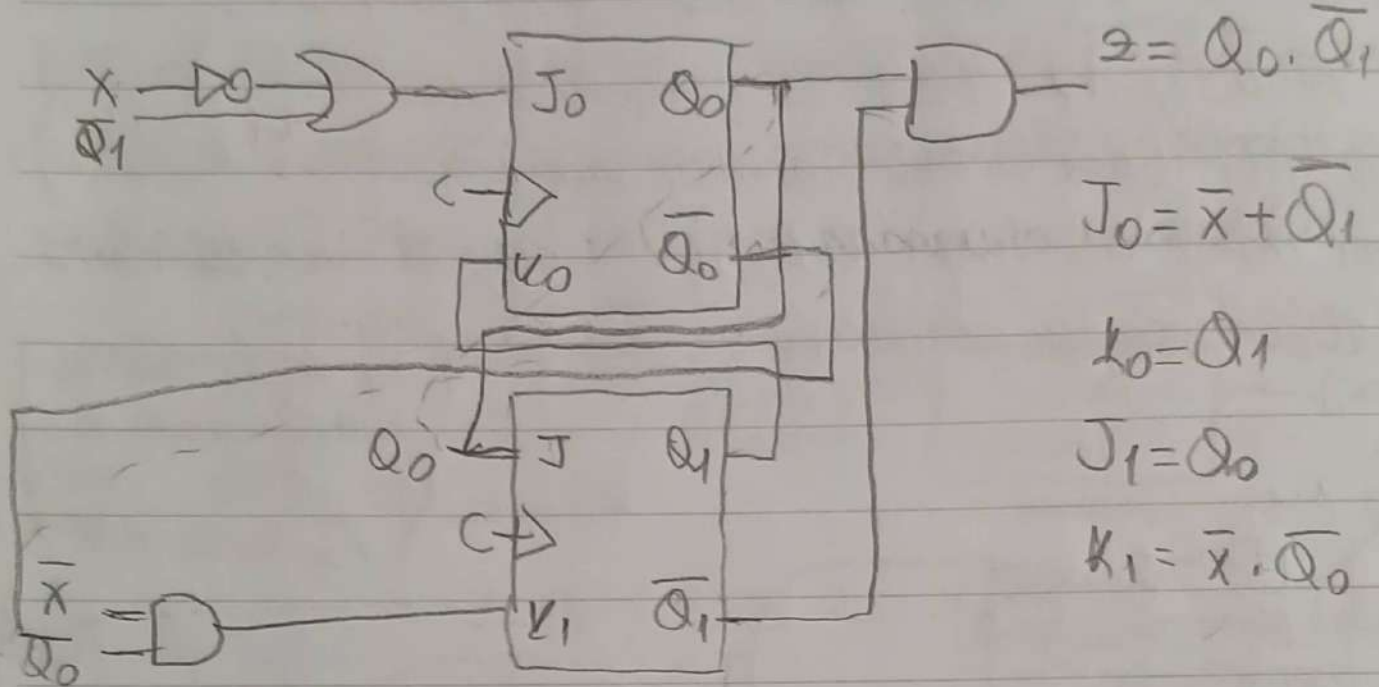
Uygun çözüm şöyle olacaktır:



Gösterildiği gibi fonksiyonların tersini sigortaların farklı yerlerde olmasından faydalanarak daha verimli çözümler üretebiliriz.

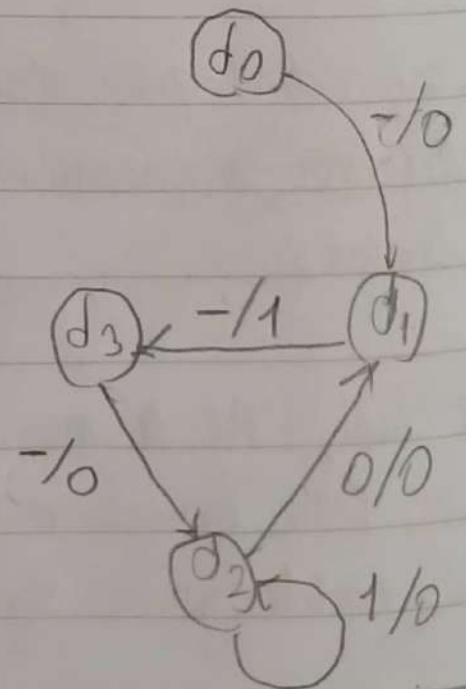
★ Yani, PROM min. terimler kanonik biçimde verilen ifadeler için, PAL indirgenmiş ifadeler için uygundur. PLA ise iki farklı yerde sigorta bulundurarak daha esnek çözümler sunar.

Ardışıl Devre Analizi (Devam)



Simgesel Durum d_x	Simdiki Durum $Q_1 Q_0$	Flip Flop Girişleri				Bir Sonraki Durumlar				$Z = Q_0 \overline{Q_1}$
		$X=0$		$X=1$		$X=0$		$X=1$		
		J_1	K_1	J_0	K_0	Q_1^+	Q_0^+	Q_1^+	Q_0^+	
d_0	0 0	0	1	1	0	0	1	0	1	0
d_1	0 1	1	0	1	0	1	1	1	1	1
d_2	1 0	0	1	1	1	0	1	1	0	0
d_3	1 1	1	0	1	1	1	0	1	0	0

Giriş	G çıkış	Simdiki Durum d_x	Bir Sonraki Durumlar		G çıkış
JK	$Q\overline{Q}$		$X=0$	$X=1$	Z
00	$Q\overline{Q}$	d_0	d_1	d_1	0
01	$Q\overline{Q}$	d_1	d_3	d_3	1
10	$Q\overline{Q}$	d_2	d_1	d_2	0
11	$Q\overline{Q}$	d_3	d_2	d_2	0



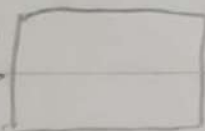
Artışlı Devre Tasarım Yöntemleri

- Devrenin ne iş yapacağı açıkça açıklanmalı.
- Durum tablosu elde edilmeli.
- Durum değişkenleri belirlenir. (FF, sayılayıcı ...)
- FF uyarma tablolarından yararlanarak FF girişlerine ait uyarma girişleri elde edilir, durum tablosuna işlenir.

ÖRNEK

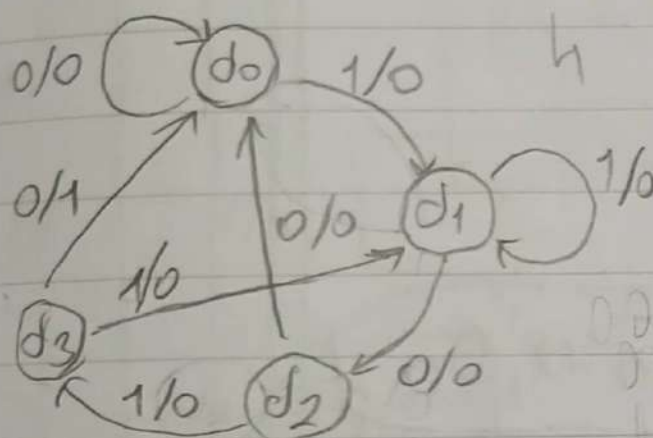
Girişi X
Çıkışı Z

1010



→ 1 alırsı halde 0.

D ff ile gerçekleştirilir.



Şimdiki Durumlar	Sonraki Durumlar		Çıkış	
	x=0	x=1	x=0	x=1
d0	d0	d1	0	0
d1	d2	d1	0	0
d2	d0	d3	0	0
d3	d0	d1	1	0

$2^k = 4$ $k=2 \rightarrow 2$ FF yeterlidir.

(uyarı tablosu!) $\begin{pmatrix} y & 0 & 0 & 0 & 1 & 1 & 0 \\ d & 0 & 1 & 1 & 0 \end{pmatrix}$

Simgesel Durum	Şimdiki Durum		Giriş	Bir Sonraki Durum		FF ₁	FF ₀	Çıkış
	Q ₁	Q ₀	X	Q ₁ ⁺	Q ₀ ⁺	d ₁	d ₀	Z
d0	0	0	0	0	0	0	0	0
d0	0	0	1	0	1	0	1	0
d1	0	1	0	1	0	1	0	0
d1	0	1	1	0	1	0	1	0
d2	1	0	0	0	0	0	0	0
d2	1	0	1	1	1	1	1	0
d3	1	1	0	0	0	0	0	1
d3	1	1	1	0	1	0	1	0

Devreyi tasarlamak için D_1 , D_0 ve Z lazım.

D_1 için

$\overline{X} \backslash Q_1 Q_0$	00	01	11	10
0		1		
1				1

$$D_1 = \overline{X}_1 \overline{Q}_1 Q_0 + X Q_1 \overline{Q}_0$$

D_0 için

$\overline{X} \backslash Q_1 Q_0$	00	01	11	10
0	0	0	0	0
1	1	1	1	1

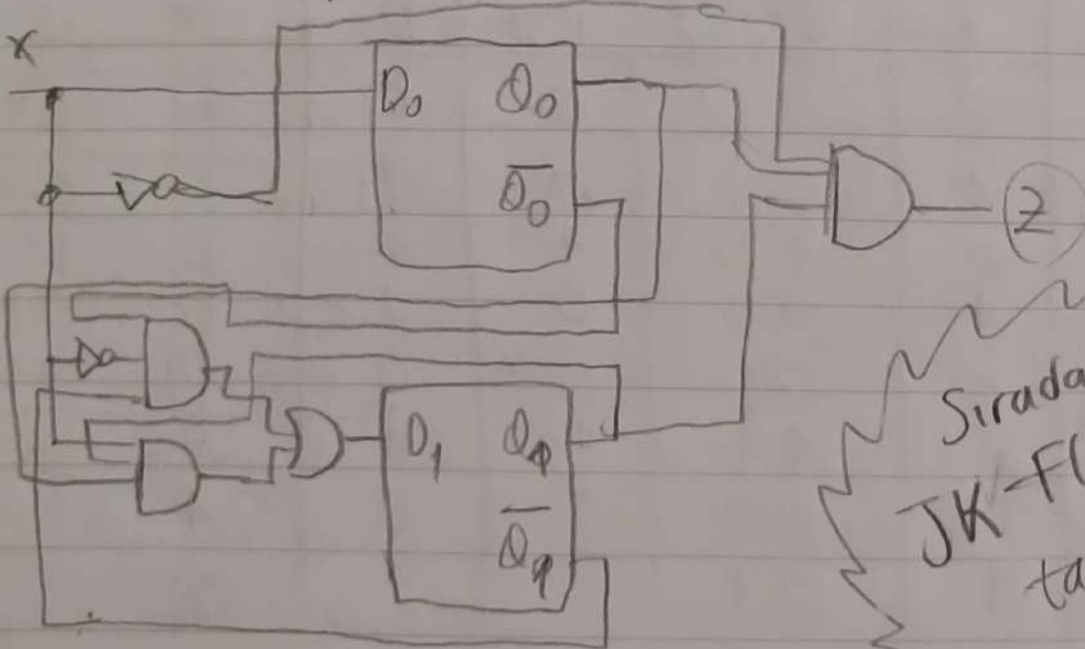
$$D_0 = X$$

Z için

$\overline{X} \backslash Q_1 Q_0$	00	01	11	10
0	0	0	1	0
1	0	0	0	0

$$Z = \overline{X} Q_1 Q_0$$

Şimdi devreyi oluşturabiliriz



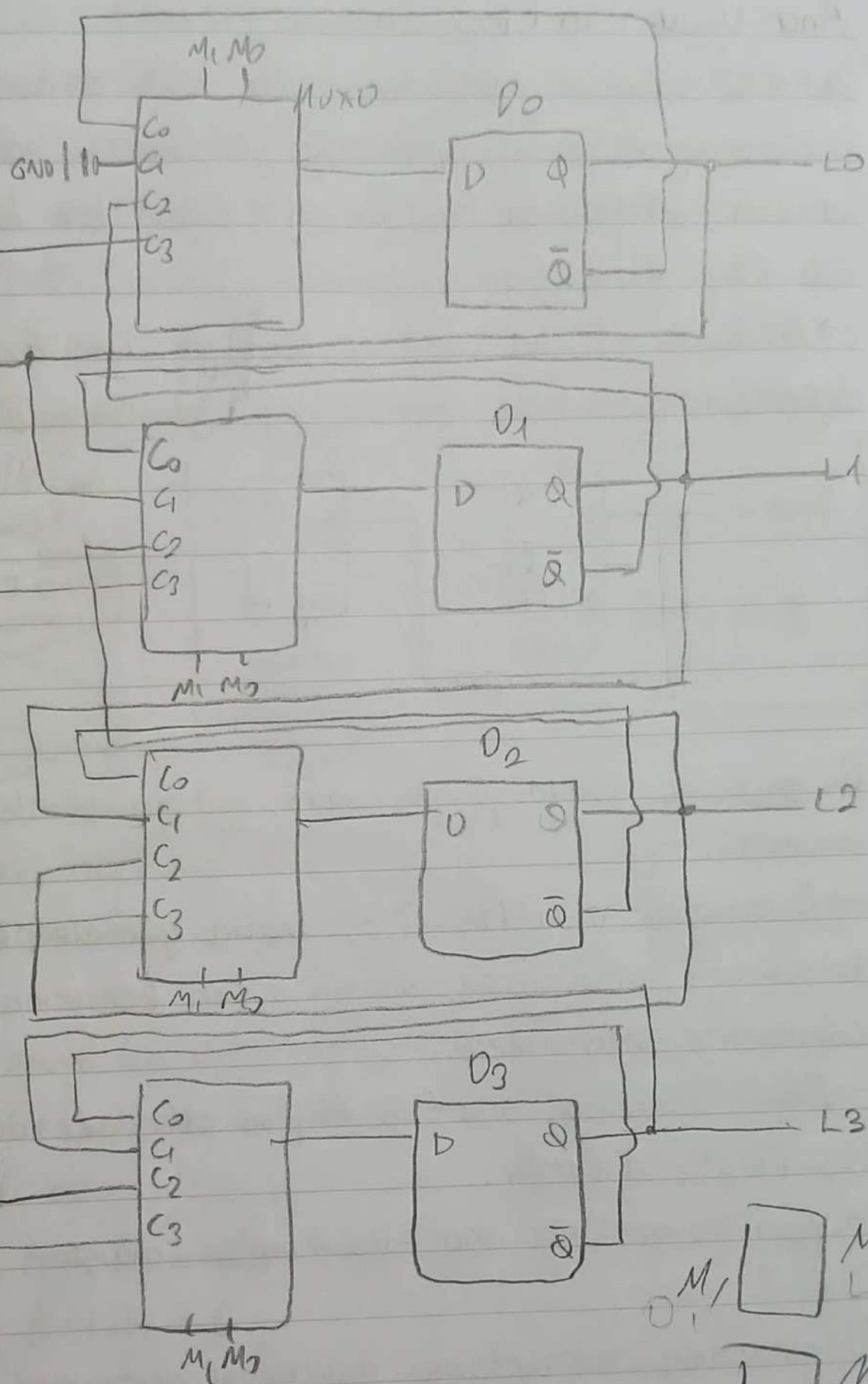
Sıradaki Dersle
JK-Flip Flop ile
tasarlanacak
(101011)

$M_1 M_0$ Memory

CLK = 1
PR

CLK → Pulse Switches

$P_3 P_2 P_1 P_0$



$M_1 M_0$

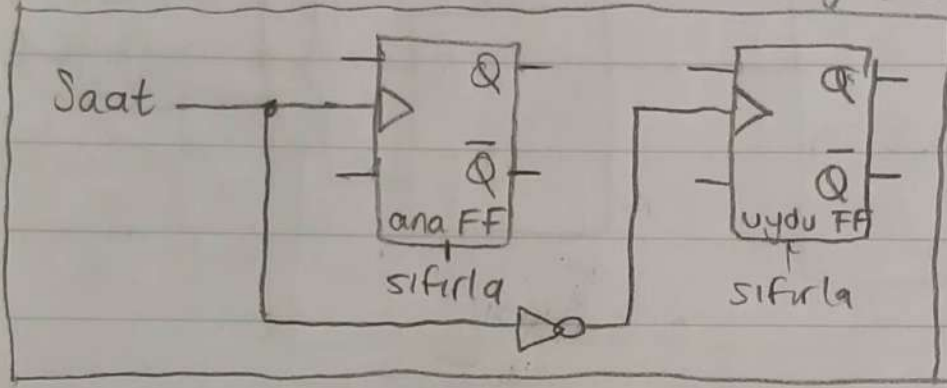
$M_3 M_2$

$P_3 P_2 P_1 P_0 M_1 M_0 G G R PR$

Ana-Uydu Flip Flop (master-slave)

İki Flip-Flop'un bağlanmasıyla elde edilen ve tetikleme periyodu içerisinde girişindeki yeni değeri alırken çıkışındaki bir önceki değeri bir süre tutan flip-flop düzenegidir.

→ İki tane FF'dan oluştuğu için girişlere bağlı yeni çıkış değerleri tetikleme anında değil, gecikmeyle yansıtılır.



tipik bir
(Ana-Uydu
Flip Flop
mimarisı.)

Bu modelde görüldüğü gibi saat, uyduya temlenerek verilmiştir.

→ Dolayısıyla uydu flip-flop, saatin yükselen kenarında tetiklediği varsayılırsa, saatin düşen kenarına kadar çıkışlarını değiştirmez.

→ Düşen kenarda, ana flip-flop'un çıkışları uydu flip-flop'a aktarılır.

→ Yani bu aralıkta ana flip-flop'un çıkışları değişmez.

Uygulamada, zamanlama davranışı açısından, bu ana flip-flop gibi davranış gösteren saklama birimlerine ihtiyaç duyulmaktadır. Böylece bir saat çevriminde iki iş yapılabilmekte ve tasarım sorunları gözlemlenmemektedir.

Tutucular (Latches)

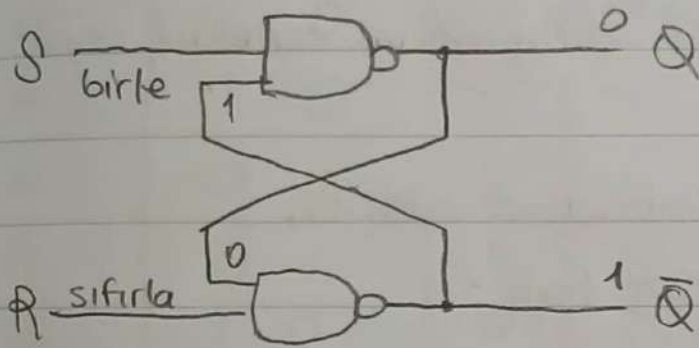
Ardışıl lojik devre tasarımında en temel saklama birimi.

Flip-Flop'lar tutuculara birkaç kapı eklenerek yapılır.

→ Bir tutucu 1 bitlik veriyi kendi üzerinde tutar.

→ VE, VEYA, TVE, TVEYA kapılarıyla kolaylıkla gerçekleştirilebilir.

Örneğin, TVE ile tutucunun yalın iç mimarisi aşağıdaki gibi gösterilebilir:



$$Q = S \cdot \overline{Q}$$

$$\Rightarrow Q = S \cdot R \cdot Q$$

$$\overline{Q} = R \cdot \overline{Q}$$

Sonuç olarak, herhangi bir andaki Q çıkışı hem bir önceki Q çıkışına hem o andaki S ve R girişlerine bağlıdır.

Örneğin; herhangi bir anda $Q(t) = 1$ olursa, bir sonraki çıkış olan $Q(t+1)$, hem S hem de R girişlerine bağlıdır. Eğer, $Q(t) = 1$ ise;

$SR = 00$ ise, $Q(t+1) = \boxed{x}$ istenmeyen durum;

$SR = 01$ ise, $Q(t+1) = 0$;

$SR = 10$ ise, $Q(t+1) = 1$;

$SR = 11$ ise, $Q(t+1) = 1 = Q(t)$;

$Q(t) = 0$ ise,

her şey aynı, sadece $SR = 11 \rightarrow Q(t+1) = 0 = Q(t)$ olur.

S	R	$Q(t)$	$Q(t+1)$	Açıklama	$Q(t+1)$
0	0	0	—	İstenmeyen durum	—
0	1	0	0	Çıkış sıfırlanır	0
1	0	0	1	Çıkış birleşir	1
1	1	0, 1	0, 1	Değişiklik yok	$Q(t)$

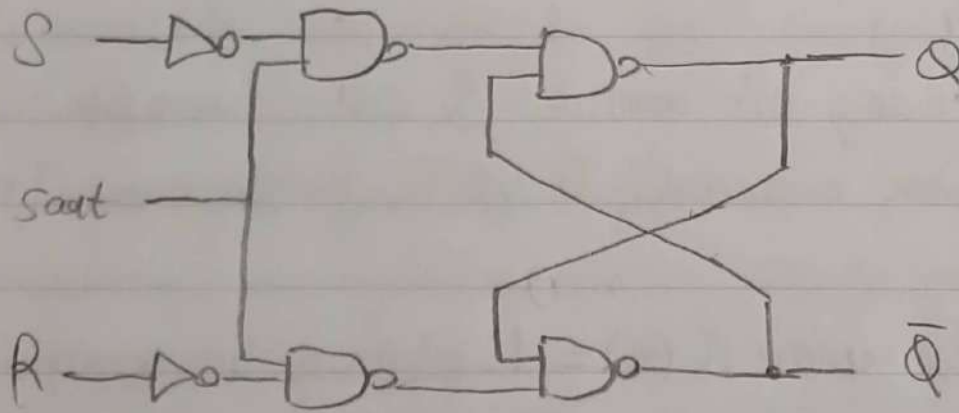
(SR tutucunun durum tablosu ve geçişler.)

Görüldüğü gibi tutucunun $(t+1)$ anındaki çıkışları t anındaki çıkışlara da bağlıdır.

→ Bu özellik devre tasarımında kolaylık ve esneklik sağlar.

Bu tutucu asenkron çalışacak mimariye sahiptir.

→ Benzer bir tutucunun senkron hali:



SENKRON
SR
TUTUCU
MİMARİSİ

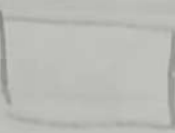
Yukarıdaki saat düzey tetiklemelidir, giriş 0 olursa çıkışlar değişmeden kalırken 1 olursa durum tablosundaki geçişler geçerli olur.

$$Q = (\overline{S} \cdot \text{saat}) \cdot \overline{Q} \quad \overline{Q} = (\overline{R} \cdot \text{saat}) \cdot Q \Rightarrow Q = (S \cdot \text{saat}) \cdot (\overline{R} \cdot \text{saat}) \cdot Q$$

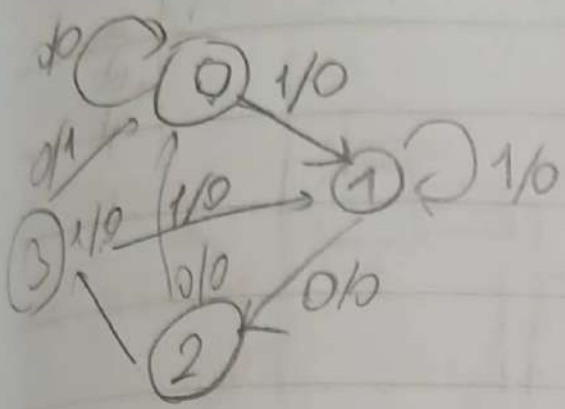
Böylece tutucuların temel yapısını ele almış olduk.

Giris x Qikisi 2

1010



alisi halde 0



	x=0	x=1	x=0 y=1
d ₀	d ₀	d ₁	0 0
d ₁	d ₂	d ₁	0 0
d ₂	d ₀	d ₃	0 0
d ₃	d ₀	d ₁	1 0

Sırası Gel Durum	Simdiki Durum Q ₁ Q ₀	Giriş X	Sonraki Durum Q ₁ ⁺ Q ₀ ⁺	Flip-Flop J ₁ K ₁ J ₀ K ₀	Çıkış Z
d ₀	0 0	0	0 0	0 0 0 0	0
d ₁	0 0	1	0 1	0 0 1 0	0
d ₂	0 1	0	1 0	1 0 0 1	0
d ₃	0 1	1	0 1	0 0 0 0	0
d ₄	1 0	0	0 0	0 1 0 0	0
d ₅	1 0	1	1 1	0 0 1 0	0
d ₆	1 1	0	0 0	0 1 0 1	1
d ₇	1 1	1	0 1	0 1 0 0	0

J₁ = $\overline{Q_1} Q_0$

$\overline{Q_1} Q_0$	00	01	11	10
X	0	1	X	X
0	0	1	X	X
1	0	0	X	X

K₁ = $Q_1 Q_0$

$Q_1 Q_0$	00	01	11	10
X	0	X	X	1
0	0	X	X	1
1	0	X	X	1

$J_1 = \overline{X} Q_0$

$K_1 = Q_0 + \overline{X}$

J₀ = $\overline{X}$

$\overline{X}$	0	1	1	0
0	0	0	0	0
1	1	0	0	1

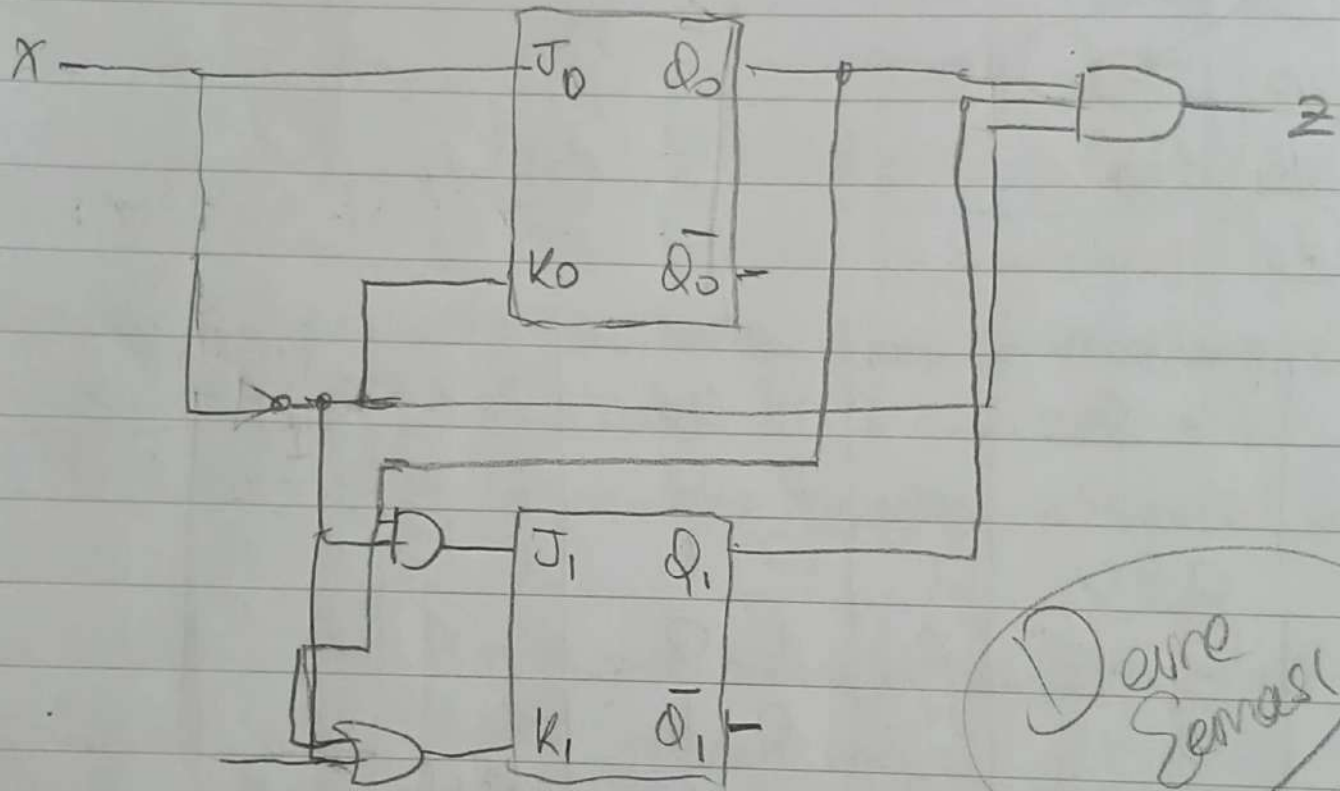
K₀ = $\overline{X}$

$\overline{X}$	0	1	1	0
0	0	1	1	0
1	0	0	0	0

$J_0 = \overline{X}$

$K_0 = \overline{X}$

$$J_1 = \bar{x}Q_0 \quad K_1 = Q_0 + x \quad J_0 = x \quad K_0 = \bar{x} \quad z = Q_1Q_0\bar{x}$$



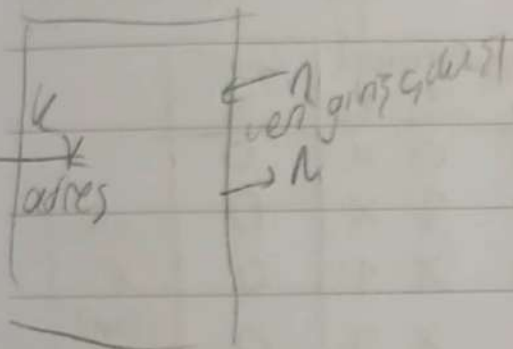
Denne
Serasi

RAM, ROM, Register

Random Access Memory
Read Only Memory

k = giriş uçları
 n = veri b/g

$2^k \times n$ bitlik
RAM



$2^k \times n$ RAM
(bit)

$2^{32} \times n \rightarrow 4GB$ Ram

$(2^{10} \times 2^{10} \times 2^{10} \times 2^4)$
4GB 4

aktif ve pasif

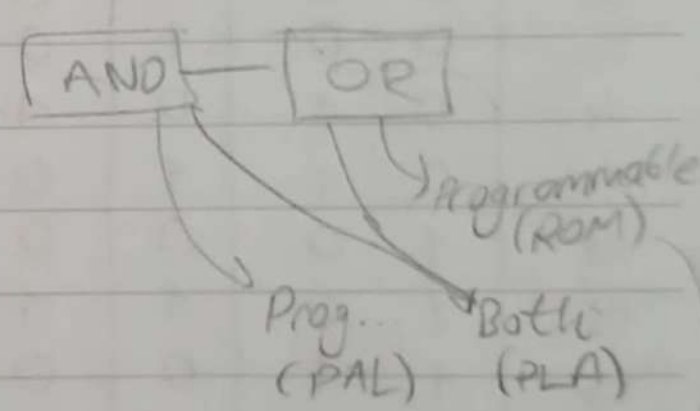
4D tipi
Flop-Flop

(Saklayıcı)

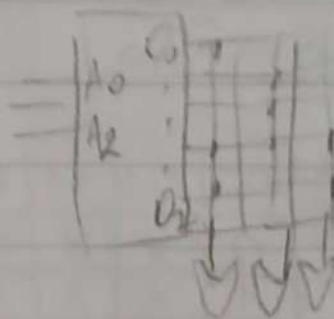
Senkroniz. (saat ile dalga)

P/PO
P/SS
S/PO
S/SS

(Senkronizasyon)



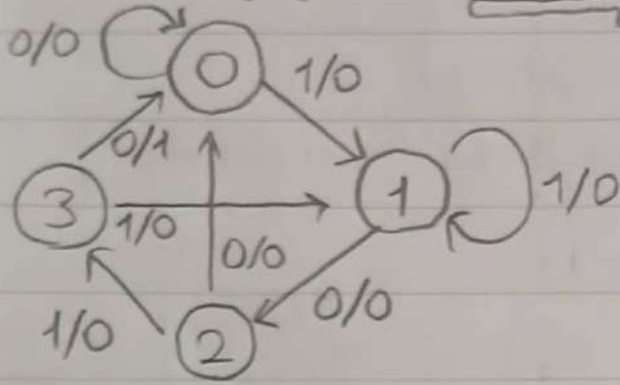
Decoder yapısı kullanılır



Aynı Örneğin JK-FlipFlop ile Çözümü

Giriş: x Çıkış: z
 Sıfır 1010 $\rightarrow$ 1, tüm diğer durumlar 0

Durum Diyagramı



Durum Tablosu

Simgesel Durum	Sonraki Durumlar		Giriş	
	$x=0$	$x=1$	$x=0$	$x=1$
d_0	d_0	d_1	0	0
d_1	d_2	d_1	0	0
d_2	d_0	d_3	0	0
d_3	d_0	d_1	1	0

[4 durum $\rightarrow 2^k=4$ $k=2$ bit ile ifade edilebilir. (2 Flip-Flop Gerek)]

Simgesel Durum	Şimdiki Durum		Giriş x	Sonraki Durum		Flip-Flop Girişleri				Çıkış z
	Q_1	Q_0		Q_1^+	Q_0^+	J_1	K_1	J_0	K_0	
d_0	0	0	0	0	0	0	X	0	X	0
d_0	0	0	1	0	1	0	X	1	X	0
d_1	0	1	0	1	0	1	X	X	1	0
d_1	0	1	1	0	1	0	X	X	0	0
d_2	1	0	0	0	0	X	1	0	X	0
d_2	1	0	1	1	1	X	0	1	X	0
d_3	1	1	0	0	0	X	1	X	1	1
d_3	1	1	1	0	1	X	1	X	0	0

X: Don't Care ($\emptyset$)

Flip-Flop Giriş ve Çıkış Fonksiyonları

J_1					K_1				
$Q_1 Q_0$	00	01	11	10	$Q_1 Q_0$	00	01	11	10
0	0	1	X	X	0	X	X	1	1
1	0	0	X	X	1	X	X	1	0

$$J_1 = \bar{x} Q_0$$

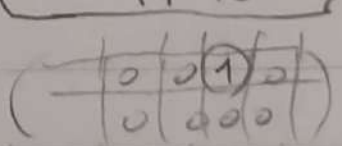
$$K_1 = \bar{x} + Q_0$$

JK-FF Uyarıma Tablosu

$Q_t Q_{t+1}$	00	01	11	10
JK	0 $\emptyset$	1 $\emptyset$	$\emptyset$ 0	$\emptyset$ 1

2 için Karnaugh'a gerek yok

$$z = Q_1 Q_0 \bar{x}$$

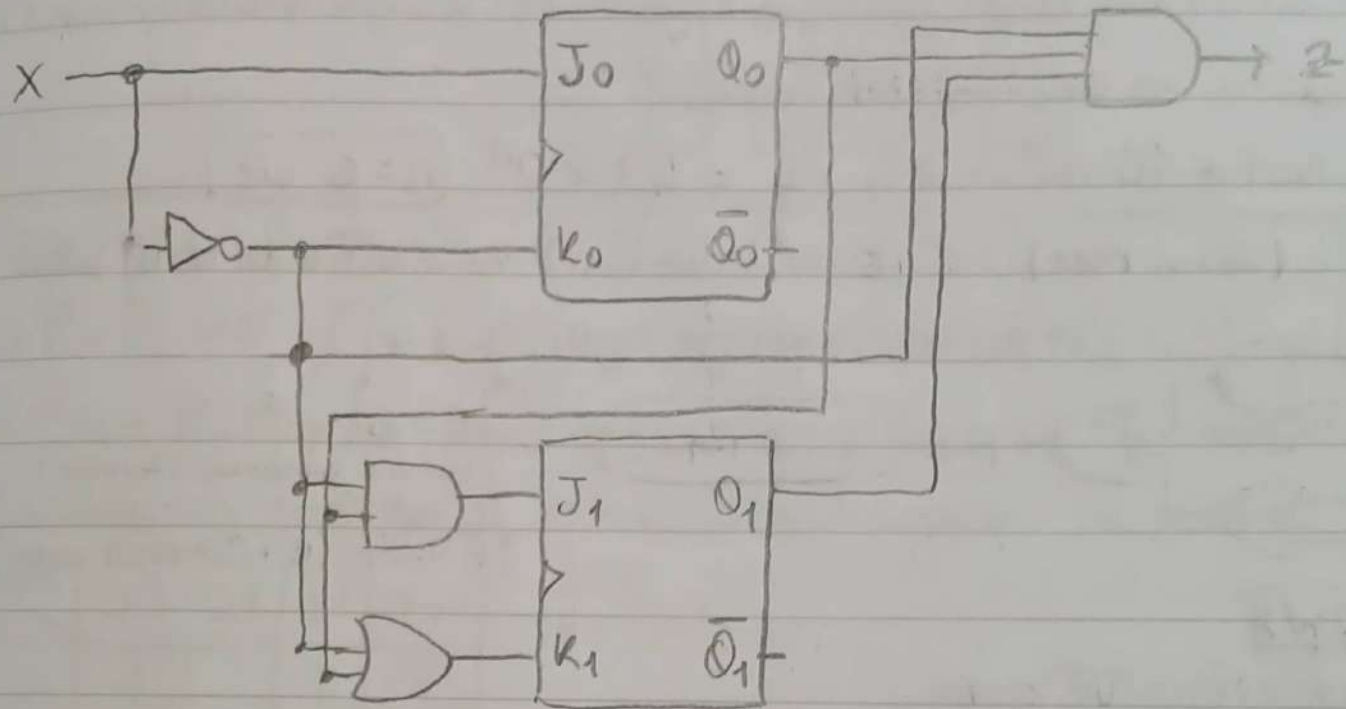


J_0					K_0				
$Q_1 Q_0$	00	01	11	10	$Q_1 Q_0$	00	01	11	10
0	0	X	X	0	0	X	1	1	X
1	1	X	X	1	1	X	0	0	X

$$J_0 = X$$

$$K_0 = \bar{x}$$

Tüm değişkenleri elde ettiğimize göre devreyi çizebiliriz:



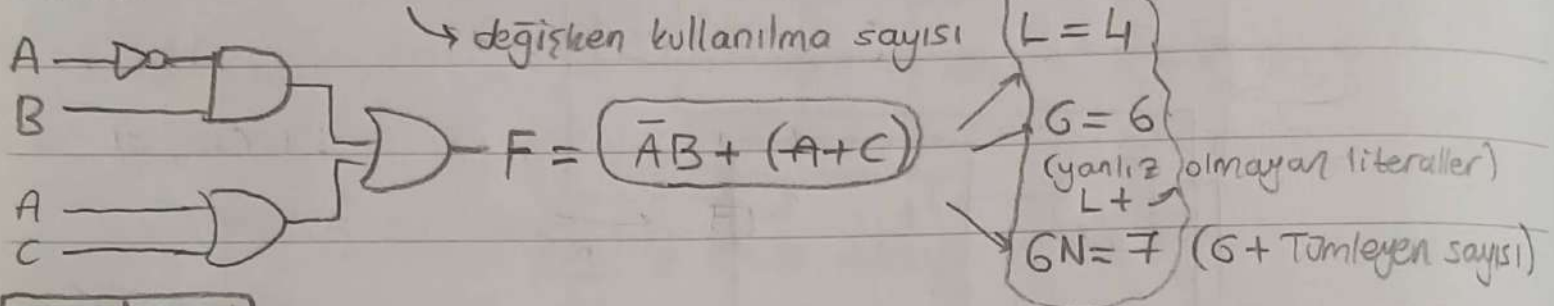
LOJİK FİNAL ÇALIŞMASI

2019 - Vize 1

Soru 1 Tek bir hat üzerinden veri haberleşmesi yapan devre lazım. Veriler alfabedeki küçük harfler, sayılar ve 4 diğer karakter (+, -, *, /) ile oluşuyorsa en az kaç bit gerekir?

→ 29 harf + 10 rakam + 4 kar. = 43 < 2ⁿ **n = 6 bit**

Soru 2 L (literal Cost), G (Gate input Cost) ve GN (G with NOTs) bulma



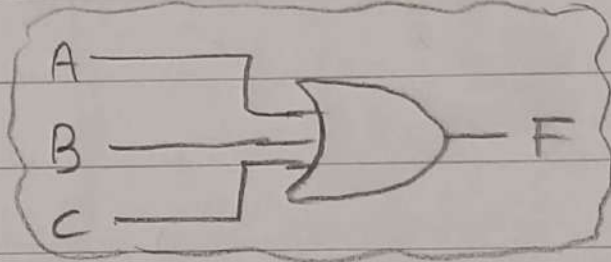
2(c) 2(d)

F'yi sadeleştirin ve çizin

$$F = \bar{A}B + A + C$$

$$= (\bar{A} + A) \cdot (B + A) + C$$

F = A + B + C

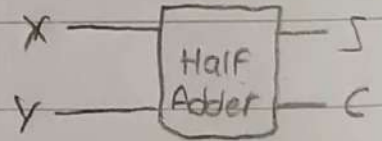
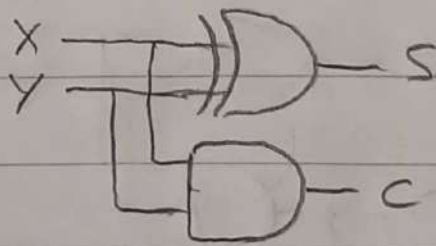


Soru 3 Yarım toplayıcı devre şeması vardır.

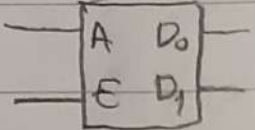
(a) Doğruluk tablosu çiziniz. (b) Kapılarla gerçekleştiriniz.

X	Y	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

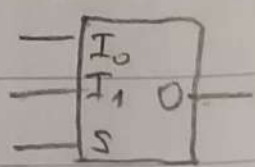
$S = X \oplus Y$ $C = XY$



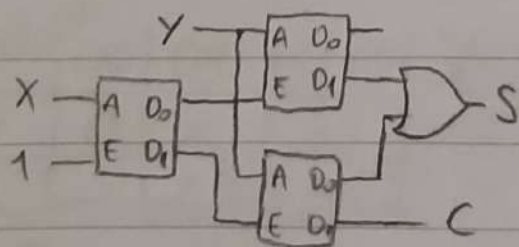
1 to 2 Decoder



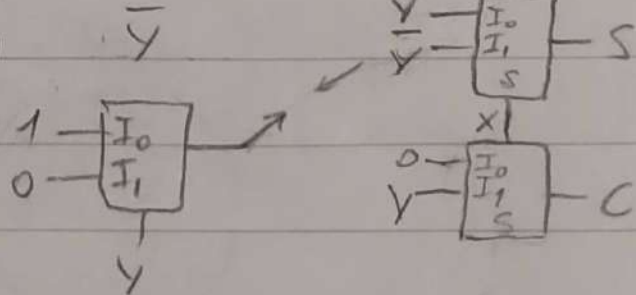
2 to 1 MUX



Decoder (1 to 2) ile gerçekleştiriniz.



2 to 1 MUX ile gerçekleştiriniz.

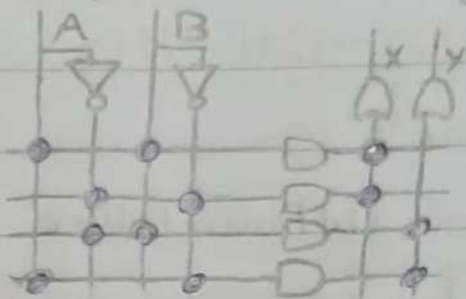


LOJİK FİNAL ÇALIŞMASI

Soru 4 PLA bağlantılarını birleştirerek X ve Y'yi gerçekleştiriniz.

$$X(A, B) = AB + \bar{A}\bar{B}$$

$$Y(A, B) = \bar{A}B + A\bar{B}$$



Soru 5 $F(A, B, C) = \sum m(0, 3, 4, 7) + d(2, 6)$ için

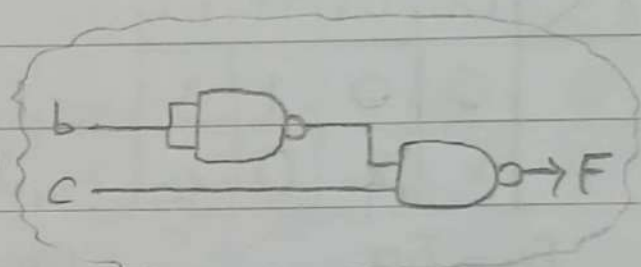
(a) $\bar{F} = ? \Rightarrow \bar{F} = \prod M(0, 3, 4, 7) + d(2, 6) = \sum m(1, 5) + d(2, 6)$

(b) F için K. Map çiz, prime implicantları göster, (c) F'i sadeleştir ve

(d) Sadeleşmiş fonksiyonu sadere NAND kapıları ile göster.

BC \ A	00	01	11	10
0	1		1	X
1	1		1	X

$$\overline{(b + \bar{c})} = (\bar{b}c)$$



$$F = \bar{c} + b$$

Soru 6 $F = A(B+C) + C + \bar{A}B\bar{C}$ ifadesini sadeleştiriniz.

$$\begin{aligned} F &= AB + AC + C + \bar{A}B\bar{C} = AB + AC + (C + \bar{A})(C + B)(C + \bar{C}) \\ &= \underbrace{AB + AC}_{A(B+C)} + \underbrace{C}_{C} + \underbrace{\bar{A}C + \bar{A}B}_{\bar{A}(B+C)} = (A + \bar{A})(B+C) + C = \boxed{B+C} \end{aligned}$$

Soru 7 $(170.625)_{10}$

(a) $(?)_2 \rightarrow 170 \Rightarrow 128 + 32 + 8 + 2 (2^7 + 2^5 + 2^3 + 2^1) \rightarrow \boxed{(10101010.101)_2}$

$$\frac{625}{1000} \rightarrow \frac{1}{2} + \frac{1}{8} (2^{-1} + 2^{-3})$$

(b) $(?)_{BCD} \rightarrow \begin{matrix} 1=0001 & 6=0110 \\ 7=0111 & 2=0010 \\ 0=0000 & 5=0101 \end{matrix} \rightarrow \boxed{(0001 \ 0111 \ 0000 \ 0110 \ 0010 \ 0101)_{BCD}}$

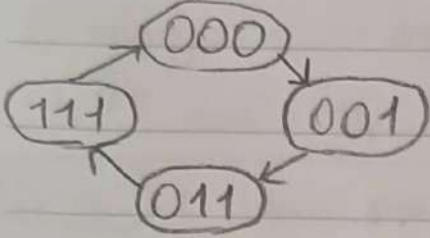
(c) $(?)_{fp(hex)} \rightarrow (432AA000)_{fp(hex)}$ (irrelevant)

LOJİK FİNAL ÇALIŞMASI

2019 Vize-2

Soru 1 $\rightarrow 000 \rightarrow 001 \rightarrow 011 \rightarrow 111$

(a) Durum Diyagramı



(b) Durum Tablosu

Şimdi			Sonra					
A	B	C	A ⁺	B ⁺	C ⁺	D _A	D _B	D _C
0	0	0	0	0	1	0	0	1
0	0	1	0	1	1	0	1	1
0	1	1	1	1	1	1	1	1
1	1	1	0	0	0	0	0	0

(c) D-FF ile

(D_A, D_B, D_C) bulunuz.
(karakteristik denklemler)

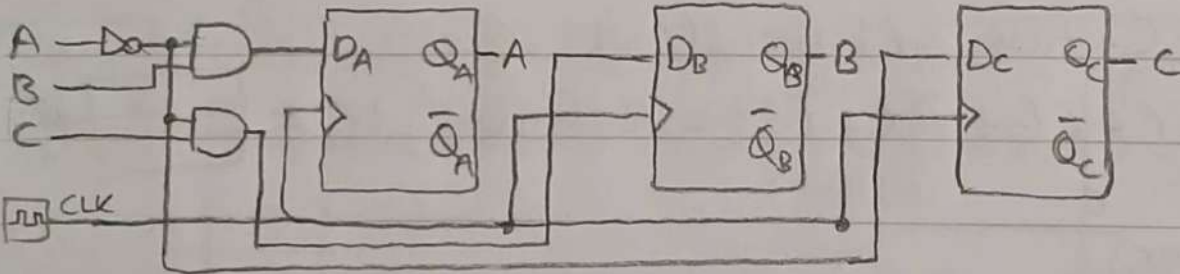
A \ BC	00	01	11	10	00	01	11	10	00	01	11	10
0	0	0	1	X	0	1	1	X	1	1	1	X
1	X	X	0	X	X	X	0	X	X	X	0	X

$$D_A = \bar{A}B$$

$$D_B = \bar{A}C$$

$$D_C = \bar{A}$$

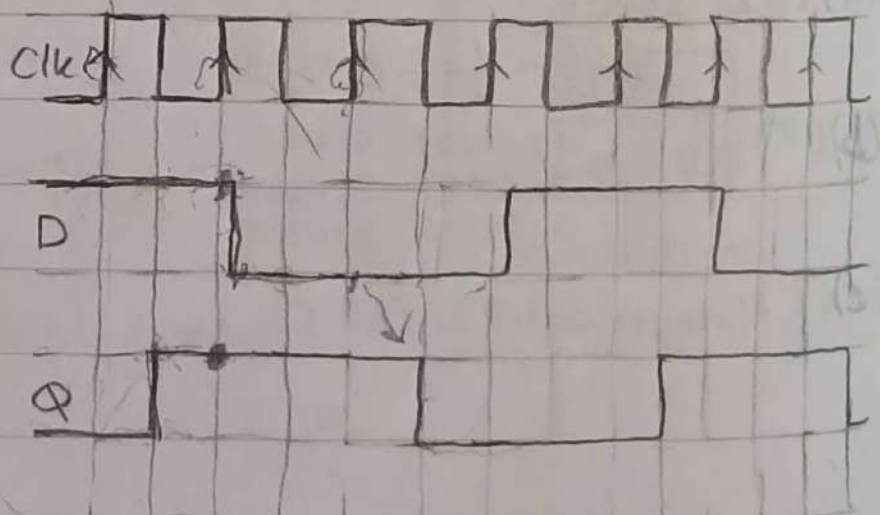
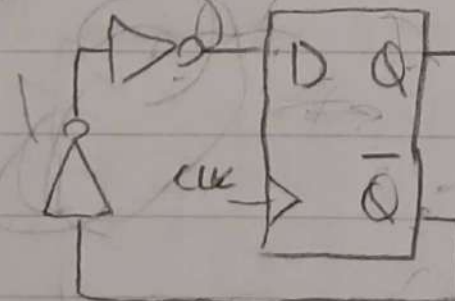
(d) Devreyi çiziniz.



SORU 2 Verilen devre

İçin grafiği tamamlayınız.

(Inverter yayılım gecikmesi: 3ns)
(D-FF yayılım gecikmesi: 5ns)



0 5 10 15 20 25 30 35 40 45 50 55 60 65